

Trabajo Práctico N° 4:
Paginación por Demanda. Algoritmos de Reemplazos.
Memoria Virtual. Entrada-Salida. Dispositivos.
Administración de Discos. Sistemas de Archivos. *Buffer*
Caché.

PARTE I: Administración de Memoria Principal.

Ejercicio 1: Memoria Virtual.

(a) *Describir qué beneficios introduce este esquema de administración de la memoria.*

Los beneficios que introduce este esquema de administración de la memoria son:

1. Aumento del espacio de direcciones disponible:
 - Cada proceso percibe un espacio de direcciones lógico mucho mayor que la memoria física real.
 - Permite ejecutar programas más grandes que la RAM disponible (“ilusión” de memoria ilimitada).
2. Protección y aislamiento entre procesos:
 - Cada proceso tiene su propio espacio de direcciones virtual, separado del resto.
 - Un proceso no puede leer ni escribir memoria de otro, aumentando seguridad y estabilidad.
3. Multiprogramación más eficiente:
 - Permite que más procesos estén cargados, parcialmente, en memoria al mismo tiempo.
 - Aumenta la utilización de la CPU y el rendimiento del sistema.
4. Carga diferencial de páginas (*paging on demand*):
 - Sólo se cargan en memoria las partes, realmente, utilizadas del programa.
 - Reduce la cantidad de RAM necesaria para ejecutar procesos.
5. Simplificación para el programador:
 - El programador no necesita gestionar, manualmente, memoria física.
 - Puede trabajar con direcciones continuas, aunque, físicamente, la memoria esté fragmentada.
6. Flexibilidad para compartir memoria:
 - Permite compartir páginas entre procesos (por ejemplo, bibliotecas dinámicas).
 - Ahorra memoria cuando varios procesos usan el mismo código.
7. Soporte para técnicas avanzadas:
 - Memoria compartida entre procesos.
 - *Copy-on-write*.
 - *Swapping*.
 - *Demand paging* y algoritmos de reemplazo.

(b) *¿En qué se debe apoyar el SO para su implementación?*

El sistema operativo, para su implementación, se debe apoyar en:

1. Unidad de Gestión de Memoria (MMU - *Memory Management Unit*):
 - Es el *hardware* encargado de traducir direcciones lógicas a físicas.
 - Realiza el mapeo mediante tablas de páginas.
 - Detecta fallos de página (*page faults*).
2. Tablas de páginas:
 - Estructuras que mantienen:
 - El marco de página en RAM (si está presente).
 - Bits de control: presente/ausente, modificado, referencia, protección.
 - Son esenciales para la traducción y para aplicar algoritmos de reemplazo.
3. Mecanismos de interrupción:
 - Interrupciones por fallo de página: Notifican al sistema operativo que una página requerida no está en RAM.
 - Permiten que el sistema operativo ejecute rutinas de manejo de páginas (traer la página desde disco, elegir víctima, etc.).
4. Memoria secundaria (disco o SSD):
 - Alberga la parte del espacio de direcciones que no cabe en RAM.
 - Incluye:
 - Área de *swap*.
 - Archivos ejecutables mapeados.
 - Sirve como respaldo para almacenar páginas ausentes.
5. Algoritmos de reemplazo de páginas:
 - El sistema operativo debe implementar criterios para decidir qué página sacar cuando la RAM está llena (FIFO, LRU, *Second Chance*, OPT -teórico-).
6. Rutinas del sistema operativo para manejo de páginas:
 - El sistema operativo debe:
 - Cargar la página desde disco.
 - Actualizar las tablas de páginas.
 - Marcar bits (modificado, referencia).
 - Reanudar el proceso interrumpido.
 - Estas rutinas constituyen el *page fault handler*.

(c) *Al implementar esta técnica utilizando paginación por demanda, las tablas de páginas de un proceso deben contar con información adicional además del marco donde se encuentra la página. ¿Cuál es esta información? ¿Por qué es necesaria?*

Cuando se implementa memoria virtual utilizando paginación por demanda, las tablas de páginas de un proceso deben contar con información adicional además del marco donde se encuentra la página. Esto se debe a que muchas páginas no están cargadas en memoria física y el sistema operativo debe poder gestionar qué páginas están presentes, cuáles fueron modificadas, etc.

Esta información es:

1. Bit de presencia/ausencia (*Present bit*):
 - Indica si la página está cargada en RAM.
 - Necesario: Sin este bit, la MMU no puede determinar si debe generar un *page fault* cuando la página aún no está en memoria.
2. Bit de modificado (*Dirty bit*):
 - Indica si la página fue escrita desde que se cargó.
 - Necesario: Si está en 1, la página debe copiarse al disco antes de ser reemplazada; si no está en 1, se puede descartar.
3. Bit de referencia (*Referenced/Accessed bit*):
 - Indica si la página fue usada recientemente.
 - Necesario: Es fundamental para los algoritmos de reemplazo (LRU aproximado, *Clock*) para decidir qué página es candidata a ser expulsada.
4. Bits de protección:
 - Especifican permisos: lectura, escritura, ejecución.
 - Necesario: Previene accesos inválidos o maliciosos; permite que un *page fault* se use también para protección.
5. Bit de validación del contenido (*Valid/Invalid*):
 - Diferencia entre:
 - Página válida, pero no presente (*swap*).
 - Página inválida (dirección que el proceso no debería usar).
 - Necesario: Evita accesos ilegales al espacio del proceso.
6. Información sobre el lugar en disco (parcial o completa):
 - Puede ser:
 - Índice del bloque en el área de *swap*.
 - Identificador en un archivo ejecutable mapeado.
 - Necesario: Para que el sistema operativo sepa dónde buscar la página cuando ocurre un *page fault*.

En resumen, utilizando paginación por demanda, cada entrada de tabla de páginas debe contener: Presente/Ausente, Modificado, Referenciado, Permisos, Validez y Ubicación en disco. Toda esta información es necesaria porque permite: generar y manejar *page faults*; decidir qué páginas reemplazar; proteger el acceso a memoria; recuperar o descartar páginas desde/hacia disco.

Ejercicio 2: Fallos de Página (Page Faults).

(a) *¿Cuándo se producen?*

Un fallo de página (*page fault*) se produce cuando un proceso intenta acceder a una página que no se encuentra cargada en la memoria física (RAM). Esto ocurre cuando la entrada de la tabla de páginas para esa página indica que no está presente (bit de presencia= 0). La MMU detecta que la dirección virtual no puede traducirse porque la página no está en RAM. Entonces, genera una interrupción al sistema operativo.

(b) *¿Quién es responsable de detectar un fallo de página?*

El responsable de detectar un fallo de página es la Unidad de Gestión de Memoria (MMU - *Memory Management Unit*).

La MMU intenta traducir la dirección virtual a una dirección física utilizando la tabla de páginas. Si la entrada correspondiente indica que la página no está presente en memoria (bit de presencia= 0) o es inválida, la MMU no puede completar la traducción y, entonces, genera una interrupción de *hardware* o *trap*. Esa interrupción se envía al sistema operativo, que, luego, se encarga de manejar el fallo (traer la página desde disco, elegir víctima, actualizar la tabla de páginas, etc.), pero la detección inicial es, puramente, responsabilidad de la MMU.

(c) *Describir las acciones que emprende el Kernel cuando se produce un fallo de página.*

Las acciones que emprende el *Kernel* cuando se produce un fallo de página son:

1. Verificar el tipo de acceso y la validez de la página:
 - El *Kernel* consulta la tabla de páginas para determinar:
 - Si la página pertenece al proceso.
 - Si el acceso es válido (lectura, escritura, ejecución).
 - Si la página, simplemente, no está presente o si el acceso es ilegal.
 - Si es un acceso ilegal, se aborta el proceso (*segmentation fault*).
2. Identificar la ubicación de la página en disco:
 - Si el acceso es válido, determina dónde se encuentra la página:
 - En el área de swap.
 - En un archivo ejecutable o de datos mapeado.
 - O si es una página nueva (*heap/stack*) a inicializar.
3. Buscar un marco libre en memoria física:
 - El *Kernel* necesita un marco para cargar la página.
 - Si hay marcos libres, usa uno.
 - Si no hay marcos libres, debe aplicar un algoritmo de reemplazo de páginas (FIFO, LRU, *Second Chance*, etc.) para elegir una “víctima”.
4. Si la página víctima está modificada, escribirla en disco:
 - Si el *dirty bit*= 1, la página debe escribirse en el área de *swap*.

- Si no está modificada, simplemente, se descarta (no hay que copiar nada).
5. Cargar la nueva página desde disco hacia RAM:
 - La página válida solicitada se copia desde:
 - *Swap*.
 - Archivo ejecutable.
 - O se inicializa si es una página nueva.
 6. Actualizar la tabla de páginas del proceso:
 - Se marca la entrada como presente.
 - Se actualiza el número de marco.
 - Se reinician los bits de referencia y modificado.
 - Se realiza cualquier actualización de protección o metadatos.
 7. Actualizar la TLB (si es necesario):
 - Si la entrada estaba en la TLB con estado inválido, se invalida y se reemplaza.
 - Puede requerir un TLB *flush* parcial o selectivo.
 8. Reanudar la ejecución del proceso:
 - Una vez que la página está disponible:
 - La instrucción interrumpida se reintenta.
 - El proceso continúa como si nada hubiera pasado.

Ejercicio 3.

Suponer que la tabla de páginas para un proceso que se está ejecutando es la que se muestra a continuación:

<i>Página</i>	<i>Bit V</i>	<i>Bit R</i>	<i>Bit M</i>	<i>Marco</i>
0	1	1	0	4
1	1	1	1	7
2	0	0	0	-
3	1	0	0	2
4	0	0	0	-
5	1	0	1	0

Asumiendo que:

- El tamaño de la página es de 512 bytes.
- Cada dirección de memoria referencia 1 byte.
- Los marcos se encuentran contiguos y en orden en memoria (0, 1, 2, ...) a partir de la dirección física 0.

¿Qué dirección física, si existe, correspondería a cada una de las siguientes direcciones virtuales? (No gestionar ningún fallo de página en caso de producirse).

(a) 1052.

Página= $1052 \text{ DIV } 512$

Página= 2.

Desplazamiento= $1052 \text{ MOD } 512$

Desplazamiento= 28.

Por lo tanto, no existe dirección física para esta dirección virtual, ya que la página 2 no está presente en memoria.

(b) 2221.

Página= $2221 \text{ DIV } 512$

Página= 4.

Desplazamiento= $2221 \text{ MOD } 512$

Desplazamiento= 173.

Por lo tanto, no existe dirección física para esta dirección virtual, ya que la página 4 no está presente en memoria.

(c) 5499.

Página= 5499 DIV 512

Página= 10.

Desplazamiento= 5499 MOD 512

Desplazamiento= 379.

Por lo tanto, no existe dirección física para esta dirección virtual, ya que la página 10 no existe en la tabla de páginas.

(d) 3101.

Página= 3101 DIV 512

Página= 6.

Desplazamiento= 3101 MOD 512

Desplazamiento= 29.

Por lo tanto, no existe dirección física para esta dirección virtual, ya que la página 6 no existe en la tabla de páginas.

Ejercicio 4.

Analizar cómo impacta el tamaño de una página (pequeña o grande) en paginación por demanda.

A continuación, se analiza cómo impacta el tamaño de una página en paginación por demanda:

1. Tamaño de página pequeño:

- Ventajas:
 - Menor fragmentación interna: Como las páginas son chicas, es menos probable que un proceso deje “espacio desperdiciado” dentro de una página.
 - Ajuste más fino entre el programa y las páginas: se cargan sólo las partes del programa necesarias; mejor uso de la memoria RAM.
 - Mayor grado de multiprogramación: Como cada proceso usa menos RAM, entran más procesos simultáneamente.
 - Mejor localización de fallos de página: Las páginas pequeñas permiten manejar mejor patrones de acceso dispersos.
- Desventajas:
 - Tablas de páginas más grandes: Como hay más páginas por proceso, la tabla crece y consume más memoria.
 - Mayor *overhead* en el TLB: más entradas necesarias; más *misses*; mayor tiempo de acceso.
 - Más fallos de páginas totales: Dado que cada página contiene menos datos, se necesita traer más páginas desde disco.

2. Tamaño de página grande:

- Ventajas:
 - Tablas de páginas más pequeñas: Menos páginas por proceso, por lo que la tabla es más corta y eficiente.
 - Menos TLB *misses*: Cada entrada cubre más direcciones, por lo que hay mayor probabilidad de aciertos en el TLB.
 - Mayor eficiencia en accesos secuenciales: Para cargas masivas o *streams* grandes, las páginas grandes reducen fallos.
 - Mejor *throughput* del disco: Cargar páginas grandes usa mejor el ancho de banda del disco.
- Desventajas:
 - Mayor fragmentación interna: Mucho espacio dentro de las páginas queda sin usar.
 - Pérdida de precisión en la asignación: Se cargan grandes bloques aunque se necesite muy poco, por lo que hay desperdicio de RAM.
 - Menor multiprogramación: Cada proceso ocupa más memoria, por lo que entran menos procesos al mismo tiempo.
 - Si la localidad es mala, empeora la *performance*: Se traen páginas enormes desde disco que no se usan.

Ejercicio 5: Asignación de Marcos a un Proceso (Conjunto de Trabajo o *Working Set*).

Con la memoria virtual paginada, no se requiere que todas las páginas de un proceso se encuentren en memoria. El SO debe controlar cuántas páginas de un proceso puede tener en la memoria principal. Existen 2 políticas que se pueden utilizar:

- *Asignación Fija.*
- *Asignación Dinámica.*

(a) *Describir cómo trabajan estas 2 políticas.*

A continuación, se describe cómo trabajan estas 2 políticas:

1. Asignación Fija:

- El sistema operativo asigna un número fijo de marcos de página a cada proceso desde el momento en que se crea. Ese número no cambia durante toda la ejecución del proceso.
- Funcionamiento:
 - Cuando el proceso inicia, el sistema operativo le asigna una cantidad determinada de marcos (por ejemplo, 10 marcos).
 - El proceso sólo puede usar esa cantidad, aunque necesite más o aunque le sobren.
 - Si el proceso necesita una página nueva y no tiene marcos libres, debe expulsar una de sus propias páginas (*page replacement local*).
 - No puede pedir marcos adicionales.
- Ventajas:
 - Simplicidad de implementación.
 - Predecible en cuanto a uso de memoria.
 - Evita que un proceso consuma demasiados marcos y afecte a otros.
- Desventajas:
 - Poco flexible.
 - Algunos procesos podrían quedar con marcos de más y otros con marcos insuficientes.
 - Aumenta el riesgo de *thrashing* si el conjunto de trabajo del proceso crece.

2. Asignación Dinámica:

- El sistema operativo asigna un número variable de marcos a cada proceso, que puede aumentar o disminuir durante su ejecución.
- Funcionamiento:
 - El sistema operativo monitorea el comportamiento del proceso (frecuencia de fallos de página, *working set*, etc.).
 - Si el proceso muestra muchos fallos de página, se le otorgan más marcos.
 - Si usa pocos marcos o su *working set* se achica, se le quitan marcos y se reasignan a otros procesos.
 - Permite balancear, dinámicamente, la memoria total entre procesos.
- Ventajas:
 - Mayor eficiencia global.
 - Se adapta a las necesidades reales del proceso.

- Reduce el riesgo de *thrashing* (cada proceso puede recibir marcos según su *working set*).
- Desventajas:
 - Mayor complejidad del sistema operativo.
 - Requiere monitorear fallos de página y comportamiento del proceso.
 - Si no está bien regulado, un proceso puede crecer demasiado y perjudicar a otros.

(b) Dada la siguiente tabla de procesos y las páginas que ellos ocupan, y teniéndose 40 marcos en la memoria principal, ¿cuántos marcos le corresponderá a cada proceso si se usa la técnica de Asignación Fija:

- con Reparto Equitativo?
- con Reparto Proporcional?

Proceso	Total de Páginas Usadas
1	15
2	20
3	20
4	8

Reparto Equitativo: Se reparten los mismos marcos a cada proceso, sin importar su tamaño.

$$\text{Marcos por proceso} = \frac{40 \text{ marcos}}{4 \text{ procesos}}$$

$$\text{Marcos por proceso} = 10.$$

Por lo tanto, si se usa la técnica de Asignación Fija con Reparto Equitativo, a cada proceso, le corresponderá 10 marcos.

Reparto Proporcional: Se reparte según la cantidad de páginas que usa cada proceso, respecto al total.

$$\text{Marcos}_{\text{proceso 1}} = 40 \frac{15}{63}$$

$$\text{Marcos}_{\text{proceso 1}} = 9,52$$

$$\text{Marcos}_{\text{proceso 1}} \cong 9.$$

$$\text{Marcos}_{\text{proceso 2}} = 40 \frac{20}{63}$$

$$\text{Marcos}_{\text{proceso 2}} = 12,7.$$

$$\text{Marcos}_{\text{proceso 2}} \cong 13.$$

$$\text{Marcos}_{\text{proceso 3}} = 40 \frac{20}{63}$$

$$\text{Marcos}_{\text{proceso 3}} = 12,7.$$

$$\text{Marcos}_{\text{proceso 3}} \cong 13.$$

$$Marcos_{proceso\ 4} = 40 \frac{8}{63}$$

$$Marcos_{proceso\ 4} = 5,08$$

$$Marcos_{proceso\ 4} \cong 5.$$

Por lo tanto, si se usa la técnica de Asignación Fija con Reparto Proporcional, a cada proceso, le corresponderá 9, 13, 13 y 5 marcos, respectivamente.

(c) ¿Cuál de los 2 repartos usados en (b) resultó más eficiente? Justificar.

Déficit por proceso= (páginas - marcos) si > 0 .

Reparto Equitativo:

Déficit por proceso=

- P1: 15 - 10= 5.
- P2: 20 - 10= 10.
- P3: 20 - 10= 10.
- P4: 8 - 10= -2.

Déficit total= 5 + 10 + 10

Déficit total= 25.

Reparto Proporcional:

Déficit por proceso=

- P1: 15 - 9= 6.
- P2: 20 - 13= 7.
- P3: 20 - 13= 7.
- P4: 8 - 5= 3.

Déficit total= 6 + 7 + 7 + 3

Déficit total= 23.

Por lo tanto, de los 2 repartos usados en (b), resultó ser más eficiente el Reparto Proporcional, ya que es mayor la probabilidad de que el conjunto de trabajo (*working set*) de cada proceso esté cubierto por sus marcos, lo que reduce fallos de página, aumenta el *throughput* y mejora la utilización de CPU.

Ejercicio 6: Reemplazo de Páginas (Selección de una Víctima).

¿Qué sucede cuando todos los marcos en la memoria principal están usados por las páginas de los procesos y se produce un fallo de página? El SO debe seleccionar una de las páginas que se encuentra en memoria principal (en un marco) como víctima, la que será reemplazada por la nueva página que produjo el fallo.

Considerar los siguientes algoritmos de selección de víctimas básicos:

- *LRU.*
- *FIFO.*
- *OPT (Óptimo).*
- *Second Chance.*

(a) *Clasificar estos algoritmos de malo a bueno de acuerdo a la tasa de fallos de página que se obtienen al utilizarlos.*

A continuación, se clasifican estos algoritmos de malo a bueno de acuerdo a la tasa de fallos de página que se obtienen al utilizarlos:

- FIFO (First-In, First-Out): Es el más simple (expulsa la página más antigua), pero puede comportarse mal frente a ciertas secuencias de referencias (sufrir la anomalía de Belady) y, por eso, suele dar la peor tasa de fallos entre los listados.
- Second Chance: Es una aproximación barata de LRU (usa el bit R para dar una “segunda oportunidad”). Normalmente, tiene rendimiento algo inferior a LRU, pero mucho mejor que FIFO en la mayoría de los casos, y con menor costo de implementación.
- LRU (Least Recently Used): Aproxima el comportamiento óptimo al expulsar la página menos recientemente usada; muy buena tasa de fallos en la práctica y es un algoritmo “*stack*” (no sufre la anomalía de Belady).
- OPT (Óptimo): Conoce el futuro y, por lo tanto, minimiza, exactamente, la cantidad de fallos de página. Es teórico (no implementable en la práctica), pero es la referencia ideal.

(b) *Analizar su funcionamiento. ¿Cómo se podrían implementar?*

1. FIFO (First-In, First-Out):

- Funcionamiento:
 - Reemplaza la página que lleva más tiempo en memoria, sin importar si fue usada recientemente o no.
 - Mantiene las páginas en una cola ordenada por tiempo de carga.
- Implementación:
 - Cola FIFO: cuando una página entra a memoria, se encola; cuando hay que reemplazar, se desencola la más antigua.
 - Muy fácil de implementar.

- No usa bits R ni M (sólo ayuda a reducir I/O si $M=1$, pero no afecta la elección de víctima).
 - Ventajas:
 - Simple, bajo costo.
 - Desventajas:
 - Mala tasa de fallos.
 - Sufre anomalía de Belady.
2. Second Chance:
- Funcionamiento:
 - Es una mejora de FIFO usando el bit R.
 - Mantiene una cola como FIFO.
 - Cuando la página más antigua tiene $R=1$, no se la reemplaza, se pone $R=0$ y se mueve al final de la cola (segunda oportunidad).
 - Se sigue avanzando en la cola hasta encontrar una página con $R=0$, que será la víctima.
 - Implementación:
 - Cola circular (implementación tipo *Clock*).
 - Requiere un bit R por página y un puntero que recorre los marcos.
 - Ventajas:
 - Mejor que FIFO.
 - Barato de implementar (sólo revisar bits R, no necesita *timestamps*).
 - Desventajas:
 - Peor que LRU.
 - No considera antigüedad real, sólo si fue usada recientemente.
3. LRU (Least Recently Used):
- Funcionamiento:
 - Reemplaza la página que hace más tiempo que no se usa.
 - Imita el comportamiento ideal: Asume que lo que no se usó recientemente es menos probable que vuelva a usarse.
 - Implementación:
 - Difícil en *hardware* puro, por lo que se aplican aproximaciones.
 - Implementación exacta (costosa): Guardar un *timestamp* cada vez que se accede a una página. Para elegir la víctima, buscar la página con *timestamp* más viejo.
 - Implementaciones aproximadas: *Clock* mejorado; NRU (*Not Recently Used*); LRU por bits (matriz de $n \times n$ bits).
 - Ventajas:
 - Excelente tasa de fallos.
 - No sufre anomalía de Belady.
 - Desventajas:
 - Implementación costosa, salvo aproximaciones.
4. OPT (Óptimo):
- Funcionamiento:
 - Expulsa la página que será usada más lejos en el futuro.
 - Implementación:
 - No se puede implementar en la vida real porque se necesitan conocer las referencias futuras.
 - Se usa: como referencia teórica para evaluar otros algoritmos; en simuladores de fallos de página.

- Ventajas:
 - Mínima tasa de fallos posible.
- Desventajas:
 - Sólo teórico.

(c) *Se sabe que la página a ser reemplazada puede estar modificada. ¿Cómo detecta el Kernel esta situación? ¿Qué acciones adicionales deben realizarse cuando sucede esta situación?*

Si la página a ser reemplazada está modificada, el *Kernel* detecta esta situación usando el bit M (*Modified*), que está en la entrada de la tabla de páginas correspondiente a esa página.

Las acciones adicionales que deben realizarse cuando sucede esta situación son:

1. Escribir la página modificada al disco (*swap*).
2. Marcar el bit M= 0.
3. Liberar el marco.

Ejercicio 7: Alcance del Reemplazo.

Al momento de tener que seleccionar una página víctima, el Kernel puede optar por 2 políticas a utilizar:

- *Reemplazo Local.*
- *Reemplazo Global.*

(a) *Describir cómo trabajan estas 2 políticas.*

A continuación, se describe cómo trabajan estas 2 políticas:

1. Reemplazo Local:

- **Funcionamiento:**
 - Cada proceso tiene asignado un conjunto fijo de marcos.
 - Ante un fallo de página, sólo puede reemplazar una página que le pertenece a él mismo, es decir, dentro de su propio conjunto de marcos.
 - No puede tomar marcos que están asignados a otros procesos.
- **Consecuencias:**
 - El desempeño del proceso depende, únicamente, de los marcos que tiene asignados.
 - Si un proceso recibe pocos marcos, puede sufrir muchos fallos de página.
 - Un proceso con baja actividad no libera marcos para otros.
 - Se evita la interferencia entre procesos.
- **Ventajas:**
 - Aísla a los procesos: Uno muy demandante no perjudica a los demás.
 - Fácil de controlar.
- **Desventajas:**
 - Puede llevar al subaprovechamiento de memoria (si un proceso no usa todos sus marcos).
 - Procesos mal asignados pueden entrar en *thrashing* sin que otros puedan ayudarlos.

2. Reemplazo Global:

- **Funcionamiento:**
 - Ante un fallo de página, el proceso puede reemplazar cualquier página del sistema, sin importar a qué proceso pertenezca.
 - La elección de víctima se hace sobre todas las páginas residentes en memoria, no sólo las del proceso actual.
- **Consecuencias:**
 - Procesos con alta demanda pueden “robar” marcos a procesos con baja demanda.
 - La memoria se usa de forma más dinámica y eficiente.
 - Pero un proceso puede afectar, gravemente, el rendimiento de otros.
- **Ventajas:**
 - Aumenta la utilización global de la memoria.
 - Se adapta, dinámicamente, a los procesos que más memoria requieren.
 - Reduce el riesgo de tener marcos ociosos.
- **Desventajas:**

- No aísla procesos: Uno puede perjudicar a los demás quitándoles marcos.
- Puede provocar *thrashing* distribuido entre procesos.
- Más complejo de controlar.

(b) *¿Es posible utilizar la política de “Asignación Fija” de marcos junto con la política de “Reemplazo Global”? Justificar.*

No, no es posible utilizar la política de “Asignación Fija” de marcos junto con la política de “Reemplazo Global”, ya que son conceptos contradictorios:

- Asignación Fija impone un límite rígido de marcos para cada proceso.
- Reemplazo Global rompe ese límite porque permite que un proceso use marcos que, originalmente, pertenecen a otro.

Si el sistema permitiera Reemplazo Global mientras dice usar Asignación Fija, dejaría de ser fija, porque un proceso podría perder marcos o ganar marcos por decisiones globales. El Reemplazo Global, por definición, requiere Asignación Dinámica.

En resumen, no es posible porque la Asignación Fija garantiza un conjunto de marcos exclusivo para cada proceso, mientras que el Reemplazo Global permite que un proceso reemplace páginas pertenecientes a otros, violando, así, la exclusividad de la Asignación Fija.

Ejercicio 8.

Considerar la siguiente secuencia de referencias de páginas:

1, 2, 15, 4, 6, 2, 1, 5, 6, 10, 4, 6, 7, 9, 1, 6, 12, 11, 12, 2, 3, 1, 8, 1, 13, 14, 15, 3, 8.

(a) Si se disponen de 5 marcos, ¿cuántos fallos de página se producirán si se utilizan las siguientes técnicas de selección de víctima? (Considerar una política de Asignación Dinámica y Reemplazo Global).

- *Second Chance.*
- *FIFO.*
- *LRU.*
- *OPT.*

Second Chance:

Marco/Página	1	2	15	4	6	2	1	5	6	10	4	6	7	9	1
F1	1	1	1	1	1	1	1*	1	1	1	4	4	4	4	4
F2		2	2	2	2	2*	2*	2	2	2	2	2	7	7	7
F3			15	15	15	15	15	5	5	5	5	5	5	9	9
F4				4	4	4	4	4	4	10	10	10	10	10	1
F5					6	6	6	6	6*	6*	6	6*	6*	6*	6*
¿PF?	X	X	X	X	X			X		X	X		X	X	X

Marco/Página	6	12	11	12	2	3	1	8	1	13	14	15	3	8	
F1	4	12	12	12*	12*	12*	12*	12	12	12	12	15	15	15	
F2	7	7	11	11	11	11	11	8	8	8	8	8	3	3	
F3	9	9	9	9	2	2	2	2	2	13	13	13	13	8	
F4	1	1	1	1	1	3	3	3	3	3	14	14	14	14	
F5	6*	6	6	6	6	6	1	1	1*	1*	1*	1	1	1	
¿PF?		X	X		X	X	X	X		X	X	X	X	X	

Queue: 1, 2, 15, 4, 6, 1, 2, 5, 10, 6, 4, 7, 9, 1, 6, 12, 11, 2, 3, 1, 12, 8, 13, 14, 1, 15, 3, 8.

Por lo tanto, si se utiliza la técnica de selección de víctima *Second Chance*, se producirán 22 fallos de página.

FIFO:

Marco/Página	1	2	15	4	6	2	1	5	6	10	4	6	7	9	1
F1	1	1	1	1	1	1	1	5	5	5	5	5	5	5	5
F2		2	2	2	2	2	2	2	2	10	10	10	10	10	10
F3			15	15	15	15	15	15	15	15	15	15	7	7	7
F4				4	4	4	4	4	4	4	4	4	4	9	9
F5					6	6	6	6	6	6	6	6	6	6	1
¿PF?	X	X	X	X	X			X		X			X	X	X

Marco/Página	6	12	11	12	2	3	1	8	1	13	14	15	3	8	
F1	6	6	6	6	6	6	1	1	1	1	1	1	3	3	
F2	10	12	12	12	12	12	12	8	8	8	8	8	8	8	
F3	7	7	11	11	11	11	11	11	11	13	13	13	13	13	
F4	9	9	9	9	2	2	2	2	2	2	14	14	14	14	
F5	1	1	1	1	1	3	3	3	3	3	3	15	15	15	
¿PF?	X	X	X		X	X	X	X		X	X	X	X		

Queue: 1, 2, 15, 4, 6, 5, 10, 7, 9, 1, 6, 12, 11, 2, 3, 1, 8, 13, 14, 15, 3.

Por lo tanto, si se utiliza la técnica de selección de víctima FIFO, se producirán 21 fallos de página.

LRU:

Marco/Página	1	2	15	4	6	2	1	5	6	10	4	6	7	9	1
F1	1	1	1	1	1	1	1	1	1	1	1	1	7	7	7
F2		2	2	2	2	2	2	2	2	2	4	4	4	4	4
F3			15	15	15	15	15	5	5	5	5	5	5	9	9
F4				4	4	4	4	4	4	10	10	10	10	10	1
F5					6	6	6	6	6	6	6	6	6	6	6
¿PF?	X	X	X	X	X			X		X	X		X	X	X

Marco/Página	6	12	11	12	2	3	1	8	1	13	14	15	3	8	
F1	7	7	11	11	11	11	11	8	8	8	8	8	3	3	
F2	4	12	12	12	12	12	12	12	12	13	13	13	13	13	
F3	9	9	9	9	2	2	2	2	2	2	14	14	14	14	
F4	1	1	1	1	1	3	3	3	3	3	3	15	15	15	
F5	6	6	6	6	6	6	1	1	1	1	1	1	1	8	
¿PF?		X	X		X	X	X	X		X	X	X	X	X	

Por lo tanto, si se utiliza la técnica de selección de víctima LRU, se producirán 22 fallos de página.

OPT:

Marco/Página	1	2	15	4	6	2	1	5	6	10	4	6	7	9	1
F1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
F2		2	2	2	2	2	2	2	2	2	2	2	2	2	2
F3			15	15	15	15	15	5	5	10	10	10	7	9	9
F4				4	4	4	4	4	4	4	4	4	4	4	4
F5					6	6	6	6	6	6	6	6	6	6	6
¿PF?	X	X	X	X	X			X		X			X	X	

Marco/Página	6	12	11	12	2	3	1	8	1	13	14	15	3	8	
F1	1	1	1	1	1	1	1	1	1	13	14	15	15	15	
F2	2	2	2	2	2	3	3	3	3	3	3	3	3	3	
F3	9	12	12	12	12	12	12	8	8	8	8	8	8	8	
F4	4	4	11	11	11	11	11	11	11	11	11	11	11	11	
F5	6	6	6	6	6	6	6	6	6	6	6	6	6	6	
¿PF?		X	X			X		X		X	X	X			

Por lo tanto, si se utiliza la técnica de selección de víctima OPT, se producirán 16 fallos de página.

(b) Suponiendo que cada atención de un fallo de página requiere de 0,1 seg. Calcular el tiempo consumido por atención a los fallos de páginas para los algoritmos de (a).

Suponiendo que cada atención de un fallo de página requiere de 0,1 seg., el tiempo consumido por atención a los fallos de páginas para los algoritmos de (a) es:

- Second Chance: $22 * 0,1 \text{ seg.} = 2,2 \text{ seg.}$
- FIFO: $21 * 0,1 \text{ seg.} = 2,1 \text{ seg.}$
- LRU: $22 * 0,1 \text{ seg.} = 2,2 \text{ seg.}$
- OPT: $16 * 0,1 \text{ seg.} = 1,6 \text{ seg.}$

Ejercicio 9.

Sean los procesos A, B y C tales que necesitan, para su ejecución, las siguientes páginas:

- A: 1, 3, 1, 2, 4, 1, 5, 1, 4, 7, 9, 4.
- B: 2, 4, 6, 2, 4, 1, 8, 3, 1, 8.
- C: 1, 2, 4, 8, 6, 1, 4, 1.

Si la secuencia de ejecución es tal que los procesos se ejecutan en la siguiente manera:

1. B demanda 2 páginas.
2. A demanda 3 páginas.
3. C demanda 2 páginas.
4. B demanda 3 páginas.
5. A demanda 3 páginas.
6. C demanda 2 páginas.
7. B demanda 2 páginas.
8. C demanda 4 páginas.
9. A demanda 3 páginas.
10. B demanda 3 páginas.
11. C termina.
12. A demanda 3 páginas.
13. B termina.
14. A termina.

(a) Considerando una política de Asignación Dinámica y Reemplazo Global y disponiéndose de 7 marcos. ¿Cuántos fallos de página se producirán si se utiliza la técnica de selección de víctima:

- LRU?
- Second Chance?

LRU:

Marcos/Página	B2	B4	A1	A3	A1	C1	C2	B6	B2	B4	A2	A4	A1	C4	C8	B1	B8
F1	B2	B2	B2	B2	B2	B2	B2	B2	B2	B2	B2	B2	B2	B2	B2	B1	B1
F2		B4	B4	B4	B4	B4	B4	B4	B4	B4	B4	B4	B4	B4	B4	B4	B8
F3			A1	A1	A1	A1	A1	A1	A1	A1	A1	A4	A4	A4	A4	A4	A4
F4				A3	A3	A3	A3	A3	A3	A3	A2	A2	A2	A2	A2	A2	A2
F5						C1	C1	C1	C1	C1	C1	C1	A1	A1	A1	A1	A1
F6							C2	C2	C2	C2	C2	C2	C2	C4	C4	C4	C4
F7								B6	B6	B6	B6	B6	B6	B6	C8	C8	C8
¿PF?	X	X	X	X		X	X	X			X	X	X	X	X	X	X

Marco/Página	C6	C1	C4	C1	A5	A1	A4	B3	B1	B8	CT	A7	A9	A4	BT	AT
F1	B1	B1	B1	B1	B1	B1	A4	A4	A4	A4	A4	A4	A4	A4	A4	
F2	B8	B8	B8	B8	B8	B8	B8	B3	B3	B3	B3	B3	B3	B3		
F3	A4	C1	C1	C1	C1	C1	C1	C1	C1	C1		A7	A7	A7	A7	
F4	C6	C6	C6	C6	C6	C6	C6	C6	B1	B1	B1	B1	B1	B1		
F5	A1	A1	A1	A1	A5	A5	A5	A5	A5	A5	A5	A5	A9	A9	A9	
F6	C4	C4	C4	C4	C4	C4	C4	C4	C4	B8	B8	B8	B8	B8		
F7	C8	C8	C8	C8	C8	A1	A1	A1	A1	A1	A1	A1	A1	A1	A1	
¿PF?	X	X			X	X	X	X	X	X		X	X			

Por lo tanto, si se utiliza la técnica de selección de víctima LRU, se producirán 24 fallos de página.

Second Chance:

Marco/Página	B2	B4	A1	A3	A1	C1	C2	B6	B2	B4	A2	A4	A1	C4	C8	B1	B8
F1	B2	B2	B2	B2	B2	B2	B2	B2	B2*	B2*	B2	B2	B2	B2	B2	B1	B1
F2		B4	B4	B4	B4	B4	B4	B4	B4	B4*	B4	B4	B4	B4	B4	B4	B8
F3			A1	A1	A1*	A1*	A1*	A1*	A1*	A1*	A1	A1	A1*	A1*	A1*	A1*	A1*
F4				A3	A3	A3	A3	A3	A3	A3	A2	A2	A2	A2	A2	A2	A2
F5						C1	C1	C1	C1	C1	C1	A4	A4	A4	A4	A4	A4
F6							C2	C2	C2	C2	C2	C2	C2	C4	C4	C4	C4
F7								B6	B6	B6	B6	B6	B6	B6	C8	C8	C8
¿PF?	X	X	X	X		X	X	X			X	X		X	X	X	X

Marco/Página	C6	C1	C4	C1	A5	A1	A4	B3	B1	B8	CT	A7	A9	A4	BT	AT
F1	B1	B1	B1	B1	B1	B1	A4	A4	A4	A4	A4	A4	A4	A4	A4	
F2	B8	B8	B8	B8	B8	B8	B8	B3	B3	B3	B3	B3	B3	B3		
F3	A1	A1	A1	A1	A1	A1*	A1*	A1*	A1*	A1*	A1*	A1*	A1*	A1*	A1*	
F4	C6	C6	C6	C6	C6	C6	C6	C6	B1	B1	B1	B1	B1	B1		
F5	A4	C1	C1	C1*	C1*	C1*	C1*	C1*	C1*	C1		A7	A7	A7	A7	
F6	C4	C4	C4*	C4*	C4	C4	C4	C4	C4	B8	B8	B8	B8	B8		
F7	C8	C8	C8	C8	A5	A5	A5	A5	A5	A5	A5	A5	A9	A9	A9	
¿PF?	X	X			X		X	X	X	X		X	X			

Queue: B2, B4, A1, A3, C1, C2, B6, B2, B4, A1, A2, A4, C4, C8, B1, B8, A1, C6, C1, C4, A5, A4, B3, A1*, B1, C1, B8, A7, A9.

Por lo tanto, si se utiliza la técnica de selección de víctima *Second Chance*, se producirán 22 fallos de página.

(b) Considerando una política de Asignación Fija con reparto equitativo y Reemplazo Local y disponiéndose de 9 marcos. ¿Cuántos fallos de página se producirán si se utiliza la técnica de selección de víctimas:

- LRU?
- Second Chance?

LRU:

Marco/Página	A1	A3	A1	A2	A4	A1	A5	A1	A4	A7	A9	A4
F1	A1	A1	A1	A1	A1	A1	A1	A1	A1	A1	A9	A9
F2		A3	A3	A3	A4	A4	A4	A4	A4	A4	A4	A4
F3				A2	A2	A2	A5	A5	A5	A7	A7	A7
¿PF?	X	X		X	X		X			X	X	

Marco/Página	B2	B4	B6	B2	B4	B1	B8	B3	B1	B8
F4	B2	B2	B2	B2	B2	B2	B8	B8	B8	B8
F5		B4	B4	B4	B4	B4	B4	B3	B3	B3
F6			B6	B6	B6	B1	B1	B1	B1	B1
¿?	X	X	X			X	X	X		

Marco/Página	C1	C2	C4	C8	C6	C1	C4	C1
F7	C1	C1	C1	C8	C8	C8	C4	C4
F8		C2	C2	C2	C6	C6	C6	C6
F9			C4	C4	C4	C1	C1	C1
¿PF?	X	X	X	X	X	X	X	

Por lo tanto, si se utiliza la técnica de selección de víctima LRU, se producirán 20 fallos de página.

Second Chance:

Marco/Página	A1	A3	A1	A2	A4	A1	A5	A1	A4	A7	A9	A4
F1	A1	A1	A1*	A1*	A1	A1*	A1*	A1*	A1*	A1	A9	A9
F2		A3	A3	A3	A4	A4	A4	A4	A4*	A4	A4	A4*
F3				A2	A2	A2	A5	A5	A5	A7	A7	A7
¿PF?	X	X		X	X		X			X	X	

Queue: A1, A3, A2, A1, A4, A5, A1, A4*, A7, A9.

Marco/Página	B2	B4	B6	B2	B4	B1	B8	B3	B1	B8
F4	B2	B2	B2	B2*	B2*	B2	B8	B8	B8	B8*
F5		B4	B4	B4	B4*	B4	B4	B3	B3	B3
F6			B6	B6	B6	B1	B1	B1	B1*	B1*
¿?	X	X	X			X	X	X		

Queue: B2, B4, B6, B2, B4, B1*, B8*, B3.

Marco/Página	C1	C2	C4	C8	C6	C1	C4	C1
F7	C1	C1	C1	C8	C8	C8	C4	C4
F8		C2	C2	C2	C6	C6	C6	C6
F9			C4	C4	C4	C1	C1	C1*
¿PF?	X	X	X	X	X	X	X	

Queue: C1, C2, C4, C8, C6, C1*, C4.

Por lo tanto, si se utiliza la técnica de selección de víctima *Second Chance*, se producirán 20 fallos de página.

Ejercicio 10.

Sean los procesos A, B y C tales que necesitan, para su ejecución, las siguientes páginas:

- *A: 1, 2, 1, 7, 2, 7, 3, 2.*
- *B: 1, 2, 5, 2, 1, 4, 5.*
- *C: 1, 3, 5, 1, 4, 2, 3.*

Si la secuencia de ejecución es tal que los procesos se ejecutan en la siguiente manera:

- 1. C demanda 1 página.*
- 2. A demanda 2 páginas.*
- 3. C demanda 1 página.*
- 4. B demanda 1 página.*
- 5. A demanda 1 página.*
- 6. C modifica la página 1.*
- 7. B demanda 2 páginas.*
- 8. A demanda 1 página.*
- 9. C demanda 1 página.*
- 10. B modifica la página 2*
- 11. A modifica la página 2.*
- 12. B demanda 2 páginas.*
- 13. A demanda 1 página.*
- 14. B demanda 2 páginas.*
- 15. C demanda 2 páginas.*
- 16. C demanda 1 página.*
- 17. A demanda 1 página.*
- 18. B termina.*
- 19. A demanda 2 páginas.*
- 20. C demanda 1 página.*
- 21. A termina.*
- 22. C termina.*

Considerando una política de Asignación Dinámica y Reemplazo Global y disponiéndose de 7 marcos, debiéndose guardar 1 marco para la gestión de descarga asincrónica de páginas modificadas, ¿cuántos fallos de página se producirán si se utiliza la técnica de selección de víctima:

- *Second Chance?*
- *FIFO?*
- *LRU?*

Second Chance:

Marco/Página	C1	A1	A2	C3	B1	A1	C1M	B2	B5	A7	C5	B2M	A2M	B2
F1	C1	C1	C1	C1	C1	C1	C1 ^{M*}	C1 ^{M*}	C1 ^M	C1 ^M	C1 ^M	C1 ^M	-	-
F2		A1	A1	A1	A1	A1*	A1*	A1*	A1	A1	A1	A1	A1	A1
F3			A2	A2	A2	A2	A2	A2	B5	B5	B5	B5	B5	B5
F4				C3	C3	C3	C3	C3	C3	A7	A7	A7	A7	A7
F5					B1	B1	B1	B1	B1	B1	C5	C5	C5	C5
F6								B2	B2	B2	B2	B2 ^{M*}	B2 ^M	B2 ^{M*}
F7	-	-	-	-	-	-	-	-	-	-	-	-	A2 ^M	A2 ^M
¿PF?	X	X	X	X	X			X	X	X	X		X	

Marco/Página	B1	A2	B4	B5	C1	C4	C2	A7	BT	A3	A2	C3	AT	CT
F1	-	-	-	-	-	-	-	-	-	-	-	-	-	-
F2	B1	B1	B1	B1	B1	C4	C4	C4	C4	C4	C4	C4	C4	C4
F3	B5	B5	B4	B4	B4	B4	C2	C2	C2	C2	C2	C2	C2	C2
F4	A7	A7	A7	B5	B5	B5	B5	A7	A7	A7	A7	A7	A7	A7
F5	C5	C5	C5	C5	C1	C1	C1	C1	C1	C1	C1	C3	C3	C3
F6	B2 ^{M*}	B2 ^{M*}	B2 ^{M*}	B2 ^{M*}	B2 ^{M*}	B2 ^M	B2 ^M	B2 ^M		A3	A3	A3		
F7	A2 ^M	A2 ^{M*}	A2 ^{M*}	A2 ^{M*}	A2 ^{M*}	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^{M*}	A2 ^{M*}		
¿PF?	X		X	X	X	X	X	X		X		X		

Queue: C1, A1, A2, C3, B1, B2, C1, A1, B5, A7, C5, B2, A2, B1, B4, B5, C1, B2, A2*, C4, C2, A7, A3, C3.

Por lo tanto, si se utiliza la técnica de selección de víctima *Second Chance*, se producirán 19 fallos de página.

FIFO:

Marco/Página	C1	A1	A2	C3	B1	A1	C1M	B2	B5	A7	C5	B2M	A2M	B2
F1	C1	C1	C1	C1	C1	C1	C1 ^M	C1 ^M	-	-	-	-	-	-
F2		A1	A1	A1	A1	A1	A1	A1	A1	A7	A7	A7	A7	A7
F3			A2	A2	A2	A2	A2	A2	A2	A2	C5	C5	C5	C5
F4				C3	C3	C3	C3	C3	C3	C3	C3	C3	A2 ^M	A2 ^M
F5					B1	B1	B1	B1	B1	B1	B1	B1	B1	B1
F6								B2	B2	B2	B2	B2 ^M	B2 ^M	B2 ^M
F7	-	-	-	-	-	-	-	-	B5	B5	B5	B5	B5	B5
¿PF?	X	X	X	X	X			X	X	X	X		X	

Marco/Página	B1	A2	B4	B5	C1	C4	C2	A7	BT	A3	A2	C3	AT	CT
F1	-	-	-	-	C1	C1	C1	C1	C1	C1	C1	C1	C1	C1
F2	A7	A7	A7	A7	A7	A7	C2	C2	C2	C2	C2	C2	C2	C2
F3	C5	C5	C5	C5	C5	C5	C5	A7	A7	A7	A7	A7	A7	A7
F4	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M		
F5	B1	B1	B4	B4	B4	B4	B4	B4		A3	A3	A3		
F6	B2 ^M	B2 ^M	B2 ^M	B2 ^M	-	-	-	-				C3	C3	
F7	B5	B5	B5	B5	B5	C4	C4	C4	C4	C4	C4	C4	C4	
¿PF?			X		X	X	X	X		X		X		

Queue: C1, A1, A2, C3, B1, B2, B5, A7, C5, A2, B4, C1, C4, C2, A7, A3, C3.

Por lo tanto, si se utiliza la técnica de selección de víctima FIFO, se producirán 17 fallos de página.

LRU:

Marco/Página	C1	A1	A2	C3	B1	A1	C1M	B2	B5	A7	C5	B2M	A2M	B2
F1	C1	C1	C1	C1	C1	C1	C1 ^M	C1 ^M	C1 ^M	C1 ^M	C1 ^M	C1 ^M	C1 ^M	C1 ^M
F2		A1	A1	A1	A1	A1	A1	A1	A1	A1	A1	A1	A2 ^M	A2 ^M
F3			A2	A2	A2	A2	A2	A2	B5	B5	B5	B5	B5	B5
F4				C3	C3	C3	C3	C3	C3	A7	A7	A7	A7	A7
F5					B1	B1	B1	B1	B1	B1	C5	C5	C5	C5
F6								B2	B2	B2	B2	B2 ^M	B2 ^M	B2 ^M
F7	-	-	-	-	-	-	-	-	-	-	-	-	-	-
¿PF?	X	X	X	X	X			X	X	X	X		X	

Marco/Página	B1	A2	B4	B5	C1	C4	C2	A7	BT	A3	A2	C3	AT	CT
F1	-	-	-	-	-	C4	C4	C4	C4	C4	C4	C4	C4	
F2	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	-	-	-	-	-	-	
F3	B5	B5	B4	B4	B4	B4	B4	B4		A3	A3	A3		
F4	A7	A7	A7	B6	B6	B6	B6	B6			A2	A2		
F5	C5	C5	C5	C5	C1	C1	C1	C1	C1	C1	C1	C3	C3	
F6	B2 ^M	B2 ^M	B2 ^M	B2 ^M	B2 ^M	-	-	A7	A7	A7	A7	A7		
F7	B1	B1	B1	B1	B1	B1	C2	C2	C2	C2	C2	C2	C2	
¿PF?	X		X	X	X	X	X	X		X	X	X		

Por lo tanto, si se utiliza la técnica de selección de víctima LRU, se producirán 20 fallos de página.

Ejercicio 11: Hiperpaginación (*Thrashing*).

(a) *¿Qué es?*

Es una situación donde el sistema pasa más tiempo realizando *swapping*/paginación entre memoria y disco que ejecutando procesos. Ocurre cuando la demanda de marcos supera, ampliamente, la memoria física disponible y el sistema operativo no puede mantener las páginas necesarias en RAM.

(b) *¿Cuáles pueden ser los motivos que la causan?*

Los motivos que la causan son:

- Multiprogramación excesiva (demasiados procesos compitiendo por pocos marcos).
- Procesos con *Working Set* grande (cada proceso necesita muchas páginas activas).
- Políticas de asignación de marcos inadecuadas (por ejemplo, asignación fija demasiado baja).
- Cambios bruscos en el patrón de acceso (localidad que cambia rápido).
- Falta general de memoria física.

(c) *¿Cómo la detecta el SO?*

El sistema operativo, normalmente, la detecta observando:

- Aumento drástico de la tasa de *page faults*.
- Caída fuerte del uso del CPU (el CPU queda ocioso esperando E/S de paginación).
- Aumento del tráfico de E/S hacia el disco por *swapping* continuo.
- *Working Set* de muchos procesos mayor a la memoria total (en modelos de *Working Set*).

Si la mayor parte del tiempo se gasta en *page faults*, hay *thrashing*.

(d) *Una vez que la detecta, ¿qué acciones puede tomar el Kernel para eliminar este problema?*

Las acciones que puede tomar el *Kernel* para eliminar este problema son:

- Reducir el grado de multiprogramación: Suspender procesos, enviarlos a *swapped out*, o no aceptar nuevos hasta estabilizar el sistema.
- Asignar más marcos a los procesos activos: Si usa políticas adaptativas (por ejemplo, PFF -*Page Fault Frequency*- o *Working Set*).

- Controlar la frecuencia de fallos de página (PFF): Si un proceso supera un umbral de *page faults*, se le asignan más marcos. Si está por debajo, se le quitan.
- Utilizar el modelo del *Working Set*: Asignar marcos suficientes para cubrir el conjunto de páginas activas del proceso.
- Reemplazo de página más eficiente: LRU, *Clock*, variantes de *WSClock*, etc.
- Priorizar procesos interactivos: Darles más marcos para evitar que se “congelen”.

Ejercicio 12.

Analizar y comparar las técnicas de PFF (Page Fault Frequency) y del Working Set. ¿Cómo las mismas permiten determinar la existencia de thrashing?

A continuación, se analizan y comparan las técnicas de PFF (*Page Fault Frequency*) y del *Working Set*:

1. PFF (*Page Fault Frequency*):

- Controla la frecuencia de fallos de página de cada proceso. Si un proceso tiene muchos fallos, necesita más marcos. Si un proceso tiene muy pocos fallos, se le pueden quitar marcos. Cada proceso debe mantener su *page fault rate* dentro de un rango aceptable.
- Esta técnica permite determinar la existencia de *thrashing* de la siguiente manera:
 - Si muchos procesos empiezan a tener frecuencias de fallos muy altas al mismo tiempo, significa que todos están compitiendo por muy pocos marcos, el disco está siendo usado constantemente y el CPU queda ocioso.
 - Entonces, el sistema operativo concluye que hay *thrashing*.
- En conclusión, esta técnica detecta *thrashing* cuando la frecuencia de fallos crece por encima de un umbral. Es una técnica reactiva, basada en el comportamiento del *page fault rate*.

2. Working Set:

- Cada proceso tiene un conjunto de páginas activas en un intervalo reciente de tiempo. Ese conjunto es su *Working Set*. El sistema operativo debe asignar, al menos, tantos marcos como páginas tenga su *Working Set*.
- Esta técnica permite determinar la existencia de *thrashing* de la siguiente manera:
 - Se suman los tamaños de los *Working Sets* de todos los procesos.
 - Si la suma es mayor que la memoria física, el sistema no puede satisfacer la demanda real de memoria.
 - Entonces, inevitablemente, ocurre *thrashing*.
- En conclusión, esta técnica detecta *thrashing* cuando la memoria no alcanza para sostener los *Working Sets* activos. Es una técnica basada en la localidad y en el uso real.

Ejercicio 13.

Considerar un sistema cuya memoria principal se administra mediante la técnica de paginación por demanda que utiliza un dispositivo de paginación, algoritmo de reemplazo global LRU y una política de asignación que reparte marcos, equitativamente, entre los procesos. El nivel de multiprogramación es, actualmente, de 4.

Analizar, para cada caso:

- *¿Qué sucede?*
- *¿Puede incrementarse el nivel de multiprogramación para aumentar el uso de la CPU?*
- *¿La paginación está siendo útil para mejorar el rendimiento del sistema?*

(a) *Uso de CPU del 13%, uso del dispositivo de paginación del 97%.*

Esto es el clásico cuadro de hiperpaginación (*thrashing*). El dispositivo de paginación está casi saturado (97%) transfiriendo páginas: la mayor parte del tiempo del sistema se consume en traer/mandar páginas al disco, mientras que el CPU está, mayormente, ocioso (13%). Con asignación equitativa, cada proceso tiene pocos marcos efectivos y provoca muchos *page faults*. El LRU global está moviendo páginas constantemente.

No, no puede incrementarse el nivel de multiprogramación para aumentar el uso de la CPU. Subir el número de procesos empeoraría la situación: habría aún más competencia por los mismos marcos, más *page faults* y mayor paginación. La acción opuesta (reducir el nivel de multiprogramación) es la indicada.

No, la paginación no está siendo útil para mejorar el rendimiento del sistema. En este escenario, la paginación está perjudicando el rendimiento (gasta E/S en mover páginas en lugar de ejecutar instrucciones útiles). Es la causa del bajo uso de la CPU.

(b) *Uso de CPU del 87%, uso del dispositivo de paginación del 3%.*

El sistema está *CPU-bound*: el procesador está ocupado (87%) y el dispositivo de paginación casi inactivo (3%). Los *page faults* son pocos y la paginación no es cuello de botella. El LRU global y la asignación actual parecen suficientes para mantener las páginas necesarias en memoria.

Sí, puede incrementarse el nivel de multiprogramación para aumentar el uso de la CPU, pero no es tan necesario porque, actualmente, el CPU está bastante ocupado. Hay margen para añadir procesos porque, actualmente, la paginación no está activa. Sin embargo, aumentar la multiprogramación puede, si se exceden los marcos libres, elevar la tasa de *page faults* y degradar rendimiento. Antes de aumentarla, conviene comprobar cuántos marcos libres quedan y, tras añadir procesos, monitorizar la tasa de *page faults*.

Sí, la paginación está siendo útil para mejorar el rendimiento del sistema. El dispositivo de paginación se usa poco porque no se necesita mucho, pero está funcionando bien (mantiene las páginas sin provocar *overhead*).

(c) *Uso de CPU del 13%, uso del dispositivo de paginación del 3%.*

Uso de CPU bajo (13%), pero uso del dispositivo de paginación también bajo (3%). Esto indica no *thrashing*. El CPU está inactivo por otras razones: procesos bloqueados en E/S no relacionado con *swapping* (por ejemplo, esperando disco de datos, red, entrada humana), espera por sincronización (*locks*) o, simplemente, no hay suficientes procesos listos para ejecutar. No hay saturación del *swap*.

Sí, puede incrementarse el nivel de multiprogramación para aumentar el uso de la CPU, pero con precaución. Al igual que en el caso anterior, hay margen para añadir procesos porque, actualmente, la paginación no está activa. Sin embargo, caben las mismas aclaraciones que en el caso anterior.

Sí, la paginación está siendo útil para mejorar el rendimiento del sistema. El dispositivo de paginación se usa poco porque no se necesita mucho, pero está funcionando bien (mantiene las páginas sin provocar *overhead*).

Ejercicio 14.

Considerar un sistema cuya memoria principal se administra mediante la técnica de paginación por demanda. Considerar las siguientes medidas de utilización:

- *Utilización del procesador: 20%.*
- *Utilización del dispositivo de paginación: 97,7%.*
- *Utilización de otros dispositivos de E/S: 5%.*

¿Cuáles de las siguientes acciones pueden mejorar la utilización del procesador?:

- *Instalar un procesador más rápido.*
- *Instalar un dispositivo de paginación mayor.*
- *Incrementar el grado de multiprogramación.*
- *Instalar más memoria principal.*
- *Decrementar el quantum para cada proceso.*

Puede mejorar la utilización del procesador instalar más memoria principal, ya que más RAM permite mantener más páginas en memoria y reduce *page faults* y la carga sobre el dispositivo de paginación: con menos paginación, el CPU podrá ejecutar más instrucciones y su utilización subirá.

Ejercicio 15.

La siguiente fórmula describe el tiempo de acceso efectivo a la memoria al utilizar paginación para la implementación de la memoria virtual:

$$TAE = A_t + [(1 - p) * A_m] + [p * (T_f + A_m)],$$

donde:

- TAE = tiempo de acceso efectivo.
- p = tasa de fallo de página ($0 \leq p \leq 1$).
- A_m = tiempo de acceso a la memoria real.
- T_f = tiempo de atención de un fallo de página.
- A_t = tiempo de acceso a la tabla de páginas. Es igual al tiempo de acceso a la memoria (A_m) si la entrada de la tabla de páginas no se encuentra en la TLB.

Suponer que se tiene una memoria virtual paginada, con tabla de páginas de 1 nivel y donde la tabla de páginas se encuentra, completamente, en la memoria.

Suponer que una falla de página tarda 300 nanosegundos si hay disponible un marco vacío o si la página reemplazada no se ha modificado, y 500 nanosegundos si se ha modificado. El tiempo de acceso a memoria es de 20 nanosegundos y el de acceso a la TLB es de 1 nanosegundo.

(a) Si se supone una tasa de fallos de página de 0,3 y que siempre se cuenta con un marco libre para atender el fallo, ¿cuál será el TAE si el 50% de las veces la entrada de la tabla de páginas se encuentra en la TLB (hit)?

$$T_f = 300 \text{ ns.}$$

$$A_m = 20 \text{ ns.}$$

$$p = 0,3.$$

$$A_t = 0,5 * 1 + (1 - 0,5) (1 + 20)$$

$$A_t = 0,5 + 0,5 * 21$$

$$A_t = 0,5 + 10,5$$

$$A_t = 11 \text{ ns.}$$

$$TAE = 11 + [(1 - 0,3) * 20] + [0,3 (300 + 20)]$$

$$TAE = 11 + (0,7 * 20) + (0,3 * 320)$$

$$TAE = 11 + 14 + 96$$

$$TAE = 121 \text{ ns.}$$

Por lo tanto, si se supone una tasa de fallos de página de 0,3 y que siempre se cuenta con un marco libre para atender el fallo, y si el 50% de las veces la entrada de la tabla de páginas se encuentra en la TLB (hit), el TAE será 121 ns.

(b) Si se supone una tasa de fallos de página de 0,3 y que el 70% de las ocasiones la página a reemplazar se encuentra modificada, ¿cuál será el TAE si el 60% de las veces la entrada de la tabla de páginas se encuentra en la TLB (hit)?

$$A_m = 20 \text{ ns.}$$

$$p = 0,3.$$

$$T_f = 0,7 * 500 + (1 - 0,7) * 300$$

$$T_f = 350 + 0,3 * 300$$

$$T_f = 350 + 90$$

$$T_f = 440 \text{ ns.}$$

$$A_t = 0,6 * 1 + (1 - 0,6) (1 + 20)$$

$$A_t = 0,6 + 0,4 * 21$$

$$A_t = 0,6 + 8,4$$

$$A_t = 9 \text{ ns.}$$

$$\text{TAE} = 9 + [(1 - 0,3) * 20] + [0,3 (440 + 20)]$$

$$\text{TAE} = 9 + (0,7 * 20) + (0,3 * 460)$$

$$\text{TAE} = 9 + 14 + 138$$

$$\text{TAE} = 161 \text{ ns.}$$

Por lo tanto, si se supone una tasa de fallos de página de 0,3 y que el 70% de las ocasiones la página a reemplazar se encuentra modificada, y si el 60% de las veces la entrada de la tabla de páginas se encuentra en la TLB (hit), el TAE será 161 ns.

(c) Si se supone que el 60% de las veces la página a reemplazar está modificada, el 100% de las veces la entrada de la tabla de páginas requerida se encuentra en la TLB (hit) y se espera un TAE menor a 200 nanosegundos, ¿cuál es la máxima tasa aceptable de fallos de página?

$$A_m = 20 \text{ ns.}$$

$$A_t = 1.$$

$$T_f = 0,6 * 500 + (1 - 0,6) * 300$$

$$T_f = 300 + 0,4 * 300$$

$$T_f = 300 + 120$$

$$T_f = 420 \text{ ns.}$$

$$200 = 1 + [(1 - p) * 20] + [p (420 + 20)]$$

$$200 = 1 + 20 - 20p + 440p$$

$$200 = 21 + 420p$$

$$420p = 200 - 21$$

$$420p = 179$$

$$p = \frac{179}{420}$$

$$p \cong 0,426.$$

Por lo tanto, si se supone que el 60% de las veces la página a reemplazar está modificada, el 100% de las veces la entrada de la tabla de páginas requerida se encuentra en la TLB (hit) y se espera un TAE menor a 200 nanosegundos, la máxima tasa aceptable de fallos de página es, aproximadamente, 0,426.

Ejercicio 16.

Considerar el siguiente programa:

```
#define Size 64
int A[Size; Size], B[Size; Size], C[Size;
Size]; int register i, j;
for (j = 0; j < Size; j++)
    for (i = 0; i < Size; i++)
        C[i; j] = A[i; j] + B[i; j];
```

Si se asume que el programa se ejecuta en un sistema que utiliza paginación por demanda para administrar la memoria, donde cada página es de 1 KB. Cada número entero (int) ocupa 4 bytes. Es claro que cada matriz requiere de 16 páginas para almacenarse. Por ejemplo: $A[0,0]..A[0,63]$, $A[1,0]..A[1,63]$, $A[2,0]..A[2,63]$ y $A[3,0]..A[3,63]$ se almacenará en la primera página.

Se asume que el sistema utiliza un working set de 4 marcos para este proceso. Uno de los 4 marcos es utilizado por el programa y los otros 3 se utilizan para datos (las matrices). También se considera que, para los índices “i” y “j”, se utilizan 2 registros, por lo que no es necesario el acceso a la memoria para estas 2 variables.

(a) Analizar cuántos fallos de páginas ocurren al ejecutar el programa (considerar las veces que se ejecuta $C[i, j] = A[i, j] + B[i, j]$).

PF= fallos por bloque * bloques por iteración j * iteraciones j
 PF= 3 * 16 * 64
 PF= 3072.

Por lo tanto, al ejecutar el programa, ocurren 3.072 fallos de página.

(b) ¿Puede ser modificado el programa para minimizar el número de fallos de páginas? En caso de ser posible, indicar la cantidad de fallos de páginas que ocurren.

Sí, el programa puede ser modificado para minimizar el número de fallos de página. En particular, cambiando el orden de los bucles para seguir el orden de almacenamiento por filas, y no por columnas, de la siguiente manera:

```
for (i = 0; i < Size; i++)
    for (j = 0; j < Size; j++)
        C[i][j] = A[i][j] + B[i][j];
```

PF= fallos por bloque * bloques
 PF= 3 * 16
 PF= 48.

Por lo tanto, en este caso, la cantidad de fallos de página que ocurren es 48.

Ejercicio 17.

Considerar las siguientes secuencias de referencias a páginas de los procesos A y B, donde se muestra el instante de tiempo en el que ocurrió cada una (1 a 78):

Proceso A																																							
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	
1	2	1	3	4	3	3	2	1	1	3	4	5	6	1	2	1	3	3	4	5	6	6	6	5	4	3	1	1	2	3	4	5	4	3	2	1	1	1	
4	4	4	4	4	4	4	4	4	4	5	5	5	5	5	5	5	5	5	5	6	6	6	6	6	5	4	3	1	1	1	1	1	1	1	1	1	1	1	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39
2	2	1	2	1	1	2	3	4	5	6	7	8	6	5	3	1	2	3	4	5	6	5	1	1	2	3	4	5	4	2	1	1	2	3	4	5	1	1	

Proceso B																																							
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	
1	2	3	3	3	4	4	5	1	1	1	2	3	4	4	4	4	1	1	2	3	6	5	6	5	4	3	1	1	1	1	4	5	1	3	2	1	1	2	
4	4	4	4	4	4	4	4	4	4	5	5	5	5	5	5	5	5	5	5	6	6	6	6	6	5	4	3	1	1	1	1	1	1	1	1	1	1	1	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39
2	2	3	3	4	4	4	3	4	5	4	6	1	1	1	2	3	3	3	4	5	5	5	1	6	2	3	1	2	3	1	1	1	2	2	2	2	3	3	

(a) Considerando una ventana $\Delta = 5$, indicar cuál sería el conjunto de trabajo de los procesos A y B en el instante 24 ($WSA(24)$ y $WSB(24)$).

Intervalo de búsqueda = [20, 21, 22, 23, 24].

$WSA(24) = \{4, 5, 6\}$.

$WSB(24) = \{2, 3, 5, 6\}$.

Por lo tanto, considerando una ventana $\Delta = 5$, el conjunto de trabajo de los procesos A y B en el instante 24 es $\{4, 5, 6\}$ y $\{2, 3, 5, 6\}$, respectivamente.

(b) Considerando una ventana $\Delta = 5$, indicar cuál sería el conjunto de trabajo de los procesos A y B en el instante 60 ($WSA(60)$ y $WSB(60)$).

Intervalo de búsqueda = [56, 57, 58, 59, 60].

$WSA(60) = \{1, 2, 3, 4, 5\}$.

$WSB(60) = \{3, 4, 5\}$.

Por lo tanto, considerando una ventana $\Delta = 5$, el conjunto de trabajo de los procesos A y B en el instante 60 es $\{1, 2, 3, 4, 5\}$ y $\{3, 4, 5\}$, respectivamente.

(c) Para el/los WS obtenidos en el inciso (a), si se cuenta con 8 frames en el sistema, ¿se puede indicar que se está ante una situación de thrashing? ¿Y si se cuenta con 6 frames?

Dados los WS obtenidos en el inciso (a) (con una demanda total de 7 frames):

- Si se cuenta con 8 frames en el sistema, no se está ante una situación de thrashing, ya que la oferta supera a la demanda.

- Si se cuenta con 6 *frames* en el sistema, se está ante una situación de *thrashing*, ya que la demanda supera a la oferta.

(d) Considerando, únicamente, el proceso A y suponiendo que al mismo se le asignaron, inicialmente, 4 marcos, donde el reemplazo de páginas es realizado considerando el algoritmo FIFO. ¿Cuál será la tasa de fallos de páginas en el instante 38 suponiendo que la misma se calcula contando los fallos de páginas que ocurrieron en las últimas 10 unidades de tiempo?

Tiempo	29	30	31	32	33	34	35	36	37	38
Marcos/Página	1	2	3	4	5	4	3	2	1	1
F1	1	1	1	1	5	5	5	5	5	5
F2	4	2	2	2	2	2	2	2	1	1
F3	5	5	3	3	3	3	3	3	3	3
F4	6	6	6	4	4	4	4	4	4	4
¿PF?		X	X	X	X				X	

$$\text{Tasa de fallos} = \frac{5}{10}$$

$$\text{Tasa de fallos} = 0,5.$$

Por lo tanto, la tasa de fallos de páginas en el instante 38, suponiendo que la misma se calcula contando los fallos de páginas que ocurrieron en las últimas 10 unidades de tiempo, es 0,5.

(e) Para el valor obtenido en el inciso (d), si se supone que el SO utiliza, como límites superior e inferior de tasa de fallos de páginas, los valores 2 y 5, respectivamente, ¿qué acción podría tomar el Kernel respecto a la cantidad de marcos asignados al proceso?

Dado que la tasa de fallos alcanza el límite superior, la acción que podría tomar el *Kernel* respecto a la cantidad de marcos asignados al proceso es diagnosticar que el proceso no tiene suficientes marcos para su conjunto de trabajo y está propenso al *thrashing*. El *Kernel* intentará aumentar la cantidad de marcos asignados al proceso A (por ejemplo, de 4 a 5) y, al aumentar el conjunto residente del proceso, se espera que la tasa de fallos caiga por debajo del límite superior, mejorando la eficiencia y evitando el *thrashing*.

PARTE II: Administración de E/S.**Ejercicio 18: Dispositivos.**

Los dispositivos, según la forma de transferir los datos, se pueden clasificar en 2 tipos:

- *Orientados a bloques.*
- *Orientados a flujos.*

(a) *Describir las diferencias entre ambos tipos.*

Las diferencias entre ambos tipos de dispositivos son:

1. Orientados a bloques:
 - Transfieren datos en bloques de tamaño fijo (por ejemplo, 512 bytes, 4 KB).
 - Acceso aleatorio: Se puede leer/escribir, directamente, en cualquier bloque sin recorrer los anteriores.
 - Son ideales para almacenamiento donde es importante acceder, rápidamente, a posiciones específicas.
 - Típicamente, gestionados por el sistema operativo como un conjunto de bloques numerados.
2. Orientados a flujos:
 - Transfieren datos como un flujo continuo de bytes, sin estructura fija.
 - Acceso secuencial: Para llegar a cierto punto del flujo, hay que pasar por los datos anteriores.
 - El sistema no maneja bloques, sino un flujo tipo “tubería” de datos.
 - Se usan para datos que se consumen o producen de manera continua.

(b) *Citar ejemplos de dispositivos de ambos tipos.*

- Ejemplos de dispositivos orientados a bloques: discos rígidos (HDD), SSD, memorias USB, tarjetas SD.
- Ejemplos de dispositivos orientados a flujos: teclado, ratón, *stream* de red, dispositivos de audio, sensores, puertos serie.

(c) *Enunciar las diferencias que existen entre los dispositivos de E/S y que el Kernel debe considerar.*

El *Kernel* debe manejar muchos tipos de dispositivos y, para hacerlo correctamente, tiene que considerar varias diferencias fundamentales entre ellos. Entre las más importantes, se encuentran:

- Velocidad de transferencia: lentos / rápidos.
- Tipo de transferencia: orientados a bloques / orientados a flujos.

- Modo de acceso: aleatorio / secuencial.
- Sincronización y temporización: síncrona / asíncrona.
- Naturaleza del dispositivo: almacenamiento / comunicación / entrada humana / sensores.
- Dirección y tamaño de datos: bytes pequeños / bloques grandes.
- Métodos de notificación: interrupciones / *polling* / DMA.
- Confiabilidad y manejo de errores: fiables / no fiables.
- Compartición: compartibles / no compartibles.
- Persistencia: datos persisten / datos no persisten.

Ejercicio 19: Técnicas de E/S.

Describir cómo trabajan las siguientes técnicas de E/S:

(a) *E/S programada.*

El CPU verifica, continuamente (*polling*), si el dispositivo está listo y copia los datos, byte a byte, entre el dispositivo y la memoria. El CPU trabaja todo el tiempo.

(b) *E/S dirigida por interrupciones.*

El CPU le pide al dispositivo que haga la operación y sigue ejecutando otras tareas. Cuando el dispositivo termina, envía una interrupción y el CPU copia los datos. El CPU trabaja sólo al principio y al final.

(c) *DMA (Acceso Directo a Memoria).*

Un módulo especial (el controlador DMA) transfiere datos entre memoria y dispositivo sin intervención del CPU. El CPU sólo inicia la operación y el DMA hace todo.

Ejercicio 20.

La técnica de E/S programa puede trabajar de dos formas:

- *E/S mapeada.*
- *E/S aislada.*

Indicar cómo trabajan estas 2 técnicas.

A continuación, se indica cómo trabajan estas 2 técnicas:

1. E/S mapeada: En esta técnica, los registros de los dispositivos de E/S (por ejemplo, “estado”, “control”, “dato”) se asignan a direcciones dentro del espacio de memoria del sistema. Esto significa que:
 - El CPU lee y escribe en los dispositivos usando las mismas instrucciones que usa para acceder a la memoria (LOAD, STORE).
 - No existe una instrucción especial para E/S.
 - El *hardware* del sistema distingue si una dirección corresponde a memoria real o a un dispositivo.
2. E/S aislada: En este caso, los dispositivos no comparten el espacio de direcciones de la memoria. Tienen un espacio separado, llamado “puertos de E/S”. Para acceder a esos puertos:
 - El CPU debe usar instrucciones especiales (por ejemplo, IN y OUT en la arquitectura x86).
 - La E/S está “aislada” y separada del acceso a memoria común.

Ejercicio 21.

Enunciar las metas que debe perseguir un SO para la administración de la entrada/salida.

Las metas que debe perseguir un sistema operativo para la administración de la entrada/salida son:

- Eficiencia: Aprovechar al máximo los dispositivos, reduciendo tiempos de espera y evitando que el CPU quede bloqueado.
- Equidad (*fairness*): Asegurar que todos los procesos tengan acceso razonable a los dispositivos, evitando que uno monopolice la E/S.
- Independencia del dispositivo: Que los programas puedan usar dispositivos sin conocer sus detalles físicos (*drivers*, diferencias de *hardware*, etc.).
- Uniformidad en la interfaz: Proveer una forma común de acceder a distintos dispositivos, simplificando el trabajo del programador.
- Concurrencia y multiplexación: Permitir que varios procesos usen dispositivos simultáneamente o que el sistema operativo intercale peticiones de forma ordenada.
- Manejo de errores: Detectar, informar y, si es posible, recuperar errores de dispositivos o transferencias de E/S.
- Protección y seguridad: Evitar accesos indebidos a dispositivos o datos y garantizar que un proceso no interfiera con otro.
- Buffering y manejo eficiente de memoria: Utilizar *buffers* para optimizar la velocidad entre CPU, memoria y dispositivos más lentos.

Ejercicio 22: Drivers.

(a) *¿Qué son?*

Los *drivers* (o controladores de dispositivo) son programas de bajo nivel que permiten que el sistema operativo se comuniquen con un dispositivo de *hardware*. Actúan como traductores entre el sistema operativo y el dispositivo, ocultando los detalles específicos del *hardware*.

(b) *¿Qué funciones mínimas deben proveer?*

Las funciones mínimas que deben proveer son:

- Inicialización del dispositivo: Configurarlos para que queden listos para operar.
- Lectura y escritura de datos: Proveer rutinas para enviar datos al dispositivo o recibirlos de él.
- Manejo de interrupciones: Responder cuando el dispositivo genera una interrupción.
- Control del dispositivo: Operaciones especiales (cambios de modo, comandos específicos, *reset*, etc.).
- Reporte y manejo de errores: Detectar fallas y comunicarlas al sistema operativo.

(c) *¿Quién determina cuáles deben ser estas funciones?*

El que determina cuáles deben ser estas funciones es el sistema operativo, que establece la API o interfaz estándar que todos los *drivers* deben implementar.

Ejercicio 23.

Realizar un gráfico que marque la relación entre el Subsistema de E/S, los drivers, los controladores de dispositivos y los dispositivos.

- El Subsistema de E/S del sistema operativo gestiona la entrada/salida de forma general.
- Los *drivers* son el puente entre el sistema operativo y el *hardware*.
- Los controladores de dispositivos son *hardware* que reciben órdenes del *driver*.
- El dispositivo es el aparato físico final.

Ejercicio 24.

Describir, mediante un ejemplo, los pasos mínimos que se suceden desde que un proceso genera un requerimiento de E/S hasta que el mismo llega al dispositivo.

Se supone que un proceso necesita leer un archivo. Los pasos mínimos que ocurren son:

1. El proceso solicita la operación: El proceso ejecuta una llamada al sistema (por ejemplo, *read()*). Esto pasa del modo usuario al modo *Kernel*.
2. El Subsistema de E/S recibe el pedido: El *Kernel* verifica qué dispositivo es, si está disponible, si necesita usar un *buffer* y si la operación requiere acceso bloqueante o no. Crea una solicitud de E/S y la coloca en la cola del dispositivo.
3. Se invoca al *driver* del dispositivo: El subsistema de E/S llama al *driver* correspondiente (por ejemplo, *driver* de disco). El *driver*: traduce el pedido del sistema operativo en comandos específicos del dispositivo; prepara los parámetros que necesita el *hardware* (sector, dirección, tamaño, etc.).
4. El *driver* ordena la operación al controlador del dispositivo: El *driver* escribe, en los registros del controlador (*hardware*), los datos necesarios (dirección del bloque a leer, cantidad de bytes, dirección de memoria donde almacenar el resultado).
5. El controlador ejecuta la operación: El controlador de disco toma los comandos y los envía al dispositivo físico. El dispositivo inicia la operación (por ejemplo, mover cabezales y leer el sector).
6. El dispositivo realiza la operación: El disco lee el bloque solicitado y, cuando termina, envía una interrupción al CPU avisando que la lectura finalizó.
7. El *driver* atiende la interrupción: El CPU detiene lo que estaba haciendo y ejecuta la rutina de interrupción del *driver*. El *driver*: confirma que la operación terminó; copia (o valida) los datos en el *buffer* del sistema operativo; marca la solicitud como completada.
8. El Subsistema de E/S devuelve el resultado al proceso: El *Kernel* despierta o reanuda al proceso que estaba esperando el dato. El proceso recibe el bloque leído y continúa ejecutando.

Ejercicio 25.

Enunciar qué servicios provee el Kernel para la administración de E/S.

Los servicios que provee el *Kernel* para la administración de E/S son:

1. Interfaz de llamadas al sistema (*syscalls*): Brinda funciones para que los procesos soliciten operaciones de E/S (*open, read, write, close*, etc.).
2. Administración de dispositivos: Identifica, inicializa y gestiona todos los dispositivos conectados.
3. Planificación de E/S: Ordena y optimiza las solicitudes de E/S (colas de pedidos, algoritmos de *scheduling*).
4. Buffering y caching: Usa *buffers* y cachés para mejorar rendimiento y reducir accesos al dispositivo.
5. Spooling: Maneja dispositivos compartidos (como impresoras), almacenando trabajos en cola.
6. Manejo de interrupciones: Atiende interrupciones generadas por los dispositivos y despierta procesos bloqueados.
7. Control y comunicación con *drivers*: Llama a los *drivers* y les pasa las solicitudes de E/S. Actúa como interfaz estándar entre ellos y los procesos.
8. Protección y seguridad: Controla qué proceso puede usar qué dispositivo y valida accesos.
9. Detección y manejo de errores de E/S: Registra, informa y trata de recuperar fallos en dispositivos o transferencias.
10. Abstracción de dispositivos: Presenta una interfaz uniforme (archivos, puertos, *sockets*), independientemente del *hardware* físico.

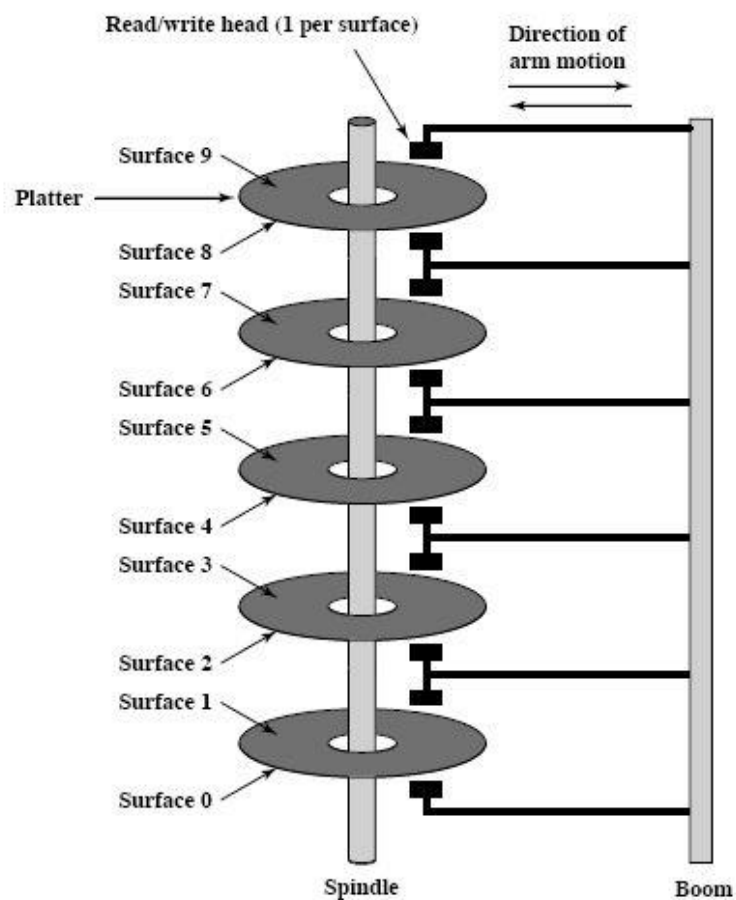
PARTE III: Administración de Discos.**Ejercicio 26.**

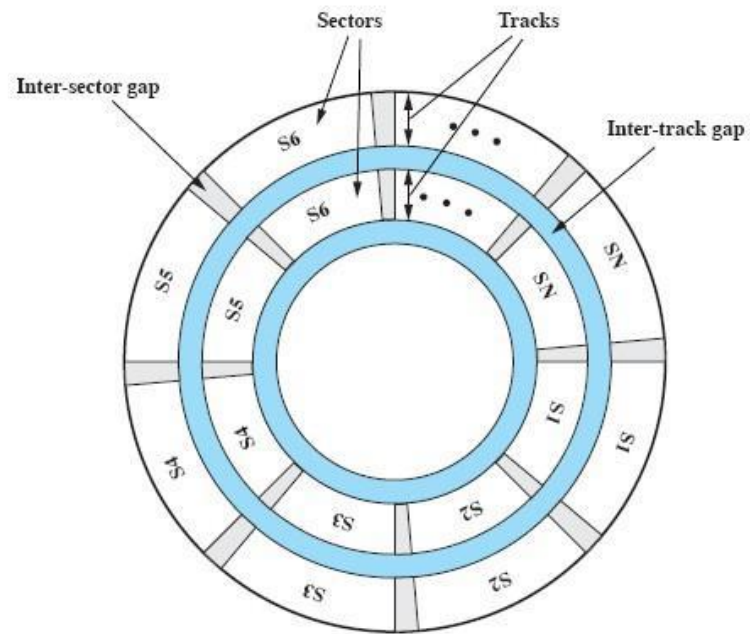
Describir, en forma sintética, cómo es la organización física de un disco. Se pueden utilizar gráficos para mayor claridad.

Un disco rígido (HDD) está formado por uno o más platos metálicos que giran a gran velocidad. Cada plato tiene dos superficies donde se puede guardar información.

La organización física es jerárquica:

1. Platos: Son discos circulares donde se almacena la información.
2. Pistas (*tracks*): Cada plato está dividido en círculos concéntricos llamados pistas.
3. Sectores: Cada pista se divide en segmentos llamados sectores. Son la unidad mínima física de lectura (típicamente, 512 bytes o 4 KB).
4. Cilindros: Son el conjunto de pistas alineadas, verticalmente, en todos los platos.
5. Cabezal (*head*): Cada superficie del plato tiene un cabezal que lee y escribe los datos. Todos los cabezales se mueven en conjunto, por lo cual se usan los cilindros como unidad lógica importante.





Ejercicio 27.

La velocidad promedio para la obtención de datos de un disco está dada por la suma de los siguientes tiempos:

- *Seek Time.*
- *Latency Time.*
- *Transfer Time.*

Dar una definición para cada uno.

La velocidad promedio para la obtención de datos de un disco está dada por la suma de los siguientes tiempos:

1. *Seek Time* (posicionamiento): Es el tiempo que tarda el cabezal del disco en moverse hasta la pista donde se encuentran los datos. Es el componente más lento y depende de la distancia que debe recorrer el cabezal.
2. *Latency Time* (latencia): Es el tiempo que tarda el disco en girar hasta que el sector deseado quede debajo del cabezal. Depende de la velocidad de rotación del disco (RPM).
3. *Transfer Time*: Es el tiempo necesario para leer o escribir los datos una vez que el cabezal ya está en el lugar correcto. Es el más rápido de los tres y depende del ancho de banda del disco.

Ejercicio 28.

Suponer un disco con las siguientes características:

- 7 platos con 2 caras utilizables cada uno.
- 1.100 cilindros, con 300 sectores por pista, donde cada sector es de 512 bytes.
- Seek time de 10 ms.
- 9.000 RPM.
- Velocidad de transferencia de 10 MiB/s (MebiBytes por segundo).

(a) *Calcular la capacidad total del disco.*

Capacidad= 512 bytes * sectores/pista * cilindros * cabezas

Capacidad= 512 bytes * 300 * 1100 * 14

Capacidad= 2365440000 bytes.

Por lo tanto, la capacidad total del disco es 2.365.440.000 bytes.

(b) *¿Cuántos sectores de disco se requerirían si se quiere almacenar 3 MiB (MebiBytes) de datos?*

3 MiB= 3 * 1048576 bytes

3 MiB= 3145728 bytes.

Sectores= $\frac{3145728 \text{ bytes}}{512 \text{ bytes}}$

Sectores= 6144.

Por lo tanto, si se quiere almacenar 3 Mib (MebiBytes) de datos, se requerirían 6.144 sectores de disco.

(c) *Calcular el tiempo de latencia.*

Latencia= $0,5 \frac{1}{RPMs}$ ms

Latencia= $0,5 \frac{1}{\frac{9000}{60*1000}}$ ms

Latencia= $0,5 \frac{1}{\frac{9000}{60000}}$ ms

Latencia= $0,5 \frac{1}{0,15}$ ms

Latencia= $0,5 * 6, \hat{6}$ ms

Latencia= $3, \hat{3}$ ms.

Por lo tanto, el tiempo de latencia es $3, \hat{3}$ ms.

Ejercicio 29.

Originalmente, los sectores de un disco se identificaban por la terna de valores (C, H, S) esto es Cylinder-Head-Sector (Cilindro, Cabeza, Sector), con C desde 0 hasta la cantidad de cilindros menos 1, H desde 0 hasta la cantidad de cabezas lecto/grabadoras menos 1 y S desde 1 hasta la cantidad de sectores por pista.

En los '90, los controladores de disco reemplazaron la identificación de los sectores por un modelo numérico denominado LBA (Logical Block Addressing). La numeración comenzaba en 0 y alcanzaba la cantidad total de sectores menos 1. Cada sector en formato (C, H, S) se puede traducir al LBA a partir de la siguiente fórmula:

$$A = (C * N_{heads} + H) * N_{sectors} + (S - 1),$$

donde:

- A es el valor LBA a obtener.
- N_{heads} es la cantidad de cabezas lecto/grabadoras.
- $N_{sectors}$ es la cantidad de sectores por pista.

También se puede obtener el valor de la terna (C, H, S) a partir de un valor LBA con las siguientes fórmulas:

- $C = LBA \text{ DIV } (N_{heads} * N_{sectors})$.
- $H = [LBA \text{ MOD } (N_{heads} * N_{sectors})] \text{ DIV } N_{sectors}$.
- $S = [LBA \text{ MOD } (N_{heads} * N_{sectors})] \text{ MOD } N_{sectors} + 1$.

(a) Si se tiene un disco con 1.020 cilindros, 16 cabezas y 63 sectores por pista, calcular la cantidad total de sectores que tendrá el disco y determinar el valor LBA para las siguientes ternas (C, H, S):

Sectores= sectores/pista * cilindros * cabezas
 Sectores= 63 * 1020 * 16
 Sectores= 1028160.

(i) (3, 2, 1).

$A = (3 * 16 + 2) * 63 + (1 - 1)$
 $A = (48 + 2) * 63 + 0$
 $A = 50 * 63 + 0$
 $A = 3150 + 0$
 $A = 3150$.

(ii) (0, 0, 1).

$A = (0 * 16 + 0) * 63 + (1 - 1)$

$$A = (0 + 0) * 63 + 0$$

$$A = 0 * 63 + 0$$

$$A = 0 + 0$$

$$A = 0.$$

(iii) (1019, 15, 63).

$$A = (1019 * 16 + 15) * 63 + (63 - 1)$$

$$A = (16304 + 15) * 63 + 62$$

$$A = 16319 * 63 + 62$$

$$A = 1028097 + 62$$

$$A = 1028159.$$

(iv) (500, 8, 44).

$$A = (500 * 16 + 8) * 63 + (44 - 1)$$

$$A = (8000 + 8) * 63 + 43$$

$$A = 8008 * 63 + 43$$

$$A = 504504 + 43$$

$$A = 504547.$$

(b) Determinar el valor de la terna (C, H, S) para los siguientes valores LBA:

(i) 1.

$$C = 1 \text{ DIV } (16 * 63)$$

$$C = 1 \text{ DIV } 1008$$

$$C = 0.$$

$$H = [1 \text{ MOD } (16 * 63)] \text{ DIV } 63$$

$$H = (1 \text{ MOD } 1008) \text{ DIV } 63$$

$$H = 1 \text{ DIV } 63$$

$$H = 0.$$

$$S = [1 \text{ MOD } (16 * 63)] \text{ MOD } 63 + 1$$

$$S = (1 \text{ MOD } 1008) \text{ MOD } 63 + 1$$

$$S = 1 \text{ MOD } 63 + 1$$

$$S = 1 + 1$$

$$S = 2.$$

$$(C, H, S) = (0, 0, 2).$$

(ii) 100.

$$C = 100 \text{ DIV } (16 * 63)$$

$$C = 100 \text{ DIV } 1008$$

$$C = 0.$$

$$H = [100 \text{ MOD } (16 * 63)] \text{ DIV } 63$$

$$H = (100 \text{ MOD } 1008) \text{ DIV } 63$$

$$H = 100 \text{ DIV } 63$$

$$H = 1.$$

$$S = [100 \text{ MOD } (16 * 63)] \text{ MOD } 63 + 1$$

$$S = (100 \text{ MOD } 1008) \text{ MOD } 63 + 1$$

$$S = 100 \text{ MOD } 63 + 1$$

$$S = 37 + 1$$

$$S = 38.$$

$$(C, H, S) = (0, 1, 38).$$

(iii) 14890.

$$C = 14890 \text{ DIV } (16 * 63)$$

$$C = 14890 \text{ DIV } 1008$$

$$C = 14.$$

$$H = [14890 \text{ MOD } (16 * 63)] \text{ DIV } 63$$

$$H = (14890 \text{ MOD } 1008) \text{ DIV } 63$$

$$H = 778 \text{ DIV } 63$$

$$H = 12.$$

$$S = [14890 \text{ MOD } (16 * 63)] \text{ MOD } 63 + 1$$

$$S = (14890 \text{ MOD } 1008) \text{ MOD } 63 + 1$$

$$S = 778 \text{ MOD } 63 + 1$$

$$S = 22 + 1$$

$$S = 23.$$

$$(C, H, S) = (14, 12, 23).$$

(iv) 45687.

$$C = 45687 \text{ DIV } (16 * 63)$$

$$C = 45687 \text{ DIV } 1008$$

$$C = 45.$$

$$H = [45687 \text{ MOD } (16 * 63)] \text{ DIV } 63$$

$$H = (45687 \text{ MOD } 1008) \text{ DIV } 63$$

$$H = 327 \text{ DIV } 63$$

$$H = 5.$$

$$S = [45687 \text{ MOD } (16 * 63)] \text{ MOD } 63 + 1$$

$$S = (45687 \text{ MOD } 1008) \text{ MOD } 63 + 1$$

$$S = 327 \text{ MOD } 63 + 1$$

$$S = 12 + 1$$

$$S = 13.$$

$$(C, H, S) = (45, 5, 13).$$

Ejercicio 30.

El Seek Time es el parámetro que posee mayor influencia en el tiempo real necesario para transferir datos desde o hacia un disco. Es importante que el Kernel planifique los diferentes requerimientos al disco para minimizar el movimiento de la cabeza lectora-grabadora.

Se analizan las diferentes políticas de planificación de requerimientos a disco con un ejemplo: Se supone un Head con movimiento en 200 tracks (numerados de 0 a 199), que está en el track 83 atendiendo un requerimiento y, anteriormente, atendió un requerimiento en el track 75.

Si la cola de requerimientos es: {86, 147, 91, 177, 94, 150, 102, 175, 130, 32, 120, 58, 66, 115}. Realizar los diagramas para calcular el total de movimientos de head para satisfacer estos requerimientos de acuerdo a los siguientes algoritmos de scheduling de discos:

(a) FCFS (First Come, First Served).

83 a 86= 3.

86 a 147= 61.

147 a 91= 56.

91 a 177= 86.

177 a 94= 83.

94 a 150= 56.

150 a 102= 48.

102 a 175= 73.

175 a 130= 45.

130 a 32= 98.

32 a 120= 88.

120 a 58= 62.

58 a 66= 8.

66 a 115= 49.

Movimientos= 3 + 61 + 56 + 86 + 83 + 56 + 48 + 73 + 45 + 98 + 88 + 62 + 8 + 49

Movimientos= 816.

(b) SSTF (Shortest Seek Time First).

83 a 86= 3.

86 a 91= 5.

91 a 94= 3.

94 a 102= 8.

102 a 115= 13.

115 a 120= 5.

120 a 130= 10.

130 a 147= 17.

147 a 150= 3.
150 a 175= 25.
175 a 177= 2.
177 a 66= 111.
66 a 58= 8.
58 a 32= 26.

Movimientos= $3 + 5 + 3 + 8 + 13 + 5 + 10 + 17 + 3 + 25 + 2 + 111 + 8 + 26$
Movimientos= 239.

(c) *Scan.*

83 a 86= 3.
86 a 91= 5.
91 a 94= 3.
94 a 102= 8.
102 a 115= 13.
115 a 120= 5.
120 a 130= 10.
130 a 147= 17.
147 a 150= 3.
150 a 175= 25.
175 a 177= 2.
177 a 199= 22.
199 a 66= 133.
66 a 58= 8.
58 a 32= 26.

Movimientos= $3 + 5 + 3 + 8 + 13 + 5 + 10 + 17 + 3 + 25 + 2 + 22 + 133 + 8 + 26$
Movimientos= 283.

(d) *Look.*

83 a 86= 3.
86 a 91= 5.
91 a 94= 3.
94 a 102= 8.
102 a 115= 13.
115 a 120= 5.
120 a 130= 10.
130 a 147= 17.
147 a 150= 3.
150 a 175= 25.
175 a 177= 2.
177 a 66= 111.
66 a 58= 8.

58 a 32= 26.

Movimientos= $3 + 5 + 3 + 8 + 13 + 5 + 10 + 17 + 3 + 25 + 2 + 111 + 8 + 26$
Movimientos= 239.

(e) *C-Scan (Circular Scan)*.

83 a 86= 3.

86 a 91= 5.

91 a 94= 3.

94 a 102= 8.

102 a 115= 13.

115 a 120= 5.

120 a 130= 10.

130 a 147= 17.

147 a 150= 3.

150 a 175= 25.

175 a 177= 2.

177 a 199= 22.

0 a 32= 32.

32 a 58= 26.

58 a 66= 8.

Movimientos= $3 + 5 + 3 + 8 + 13 + 5 + 10 + 17 + 3 + 25 + 2 + 22 + 32 + 26 + 8$
Movimientos= 182.

(f) *C-Look (Circular Look)*.

83 a 86= 3.

86 a 91= 5.

91 a 94= 3.

94 a 102= 8.

102 a 115= 13.

115 a 120= 5.

120 a 130= 10.

130 a 147= 17.

147 a 150= 3.

150 a 175= 25.

175 a 177= 2.

32 a 58= 26.

58 a 66= 8.

Movimientos= $3 + 5 + 3 + 8 + 13 + 5 + 10 + 17 + 3 + 25 + 2 + 26 + 8$
Movimientos= 128.

Ejercicio 31.

¿Alguno de los algoritmos analizados en el ejercicio anterior pueden causar inanición de requerimientos?

Sí, de los algoritmos analizados en el ejercicio anterior, SSTF puede causar inanición de requerimientos, ya que el cabezal atiende siempre el pedido más cercano y, si llegan muchos pedidos cercanos al cabezal, uno que está lejos podría nunca ser atendido.

Ejercicio 32.

Se supone un disco con 300 pistas (numerados de 0 a 299), donde la cabeza se encuentra en la pista 143 atendiendo un requerimiento y, anteriormente, atendió un requerimiento en la pista 125.

Si la cola inicial de requerimientos es: {126, 147, 81, 277, 94, 150, 212, 175, 140, 225, 280, 50, 99, 118, 22, 55} y, luego de 30 movimientos, se incorporan los requerimientos de las pistas {75, 115, 220 y 266}. Realizar los diagramas para calcular el total de movimientos de head para satisfacer estos requerimientos de acuerdo a los siguientes algoritmos de scheduling de discos:

(a) FCFS (First Come, First Served).

143 a 126= 17.

126 a 147= 21.

(se incorporan los nuevos requerimientos)

147 a 81= 66.

81 a 277= 196.

277 a 94= 183.

94 a 150= 56.

150 a 212= 62.

212 a 175= 37.

175 a 140= 35.

140 a 225= 85.

225 a 280= 55.

280 a 50= 230.

50 a 99= 49.

99 a 118= 19.

118 a 22= 96.

22 a 55= 33.

55 a 75= 20.

75 a 115= 40.

115 a 220= 105.

220 a 266= 46.

Movimientos= 17 + 21 + 66 + 196 + 183 + 56 + 62 + 37 + 35 + 85 + 55 + 230 + 49 + 19 + 96 + 33 + 20 + 40 + 105 + 46

Movimientos= 1451.

(b) SSTF (Shortest Seek Time First).

143 a 140= 3.

140 a 147= 7.

147 a 150= 3.

150 a 126= 24.

(se incorporan los nuevos requerimientos)

126 a 118= 8.

118 a 115= 3.

115 a 99= 16.
 99 a 94= 5.
 94 a 81= 13.
 81 a 75= 6.
 75 a 55= 20.
 55 a 50= 5.
 50 a 22= 28.
 22 a 175= 153.
 175 a 212= 37.
 212 a 220= 8.
 220 a 225= 5.
 225 a 266= 41.
 266 a 277= 11.
 277 a 280= 3.

Movimientos= $3 + 7 + 3 + 24 + 8 + 3 + 16 + 5 + 13 + 6 + 20 + 5 + 28 + 153 + 37 + 8 + 5$
 $+ 41 + 11 + 3$

Movimientos= 399.

(c) *Scan*.

143 a 147= 4.
 147 a 150= 3.
 150 a 175= 25.
 175 a 212= 37.
 212 a 220= 8.
 220 a 225= 5.
 225 a 266= 41.
 266 a 277= 11.
 277 a 280= 3.
 280 a 299= 19.
 299 a 140= 159.
 140 a 126= 14.
 126 a 118= 8.
 118 a 115= 3.
 115 a 99= 16.
 99 a 94= 5.
 94 a 81= 13.
 81 a 75= 6.
 75 a 55= 20.
 55 a 50= 5.
 50 a 22= 28.

(se incorporan los nuevos requerimientos)

Movimientos= $4 + 3 + 25 + 37 + 8 + 5 + 41 + 11 + 3 + 19 + 159 + 14 + 8 + 3 + 16 + 5 +$
 $13 + 6 + 20 + 5 + 28$

Movimientos= 433.

(d) Look.

143 a 147= 4.

147 a 150= 3.

150 a 175= 25.

(se incorporan los nuevos requerimientos)

175 a 212= 37.

212 a 220= 8.

220 a 225= 5.

225 a 266= 41.

266 a 277= 11.

277 a 280= 3.

280 a 140= 140.

140 a 126= 14.

126 a 118= 8.

118 a 115= 3.

115 a 99= 16.

99 a 94= 5.

94 a 81= 13.

81 a 75= 6.

75 a 55= 20.

55 a 50= 5.

50 a 22= 28.

Movimientos= $4 + 3 + 25 + 37 + 8 + 5 + 41 + 11 + 3 + 140 + 14 + 8 + 3 + 16 + 5 + 13 + 6 + 20 + 5 + 28$

Movimientos= 395.

(e) C-Scan (Circular Scan).

143 a 147= 4.

147 a 150= 3.

150 a 175= 25.

(se incorporan los nuevos requerimientos)

175 a 212= 37.

212 a 220= 8.

220 a 225= 5.

225 a 266= 41.

266 a 277= 11.

277 a 280= 3.

280 a 299= 19.

0 a 22= 22.

22 a 50= 28.

50 a 55= 5.

55 a 75= 20.

75 a 81= 6.

81 a 94= 13.

94 a 99= 5.

99 a 115= 16.

115 a 118= 3.

118 a 126= 8.

126 a 140= 14.

Movimientos= $4 + 3 + 25 + 37 + 8 + 5 + 41 + 11 + 3 + 19 + 22 + 28 + 5 + 20 + 6 + 13 + 5 + 16 + 3 + 8 + 14$

Movimientos= 296.

(f) *C-Look (Circular Look)*.

143 a 147= 4.

147 a 150= 3.

150 a 175= 25.

(se incorporan los nuevos requerimientos)

175 a 212= 37.

212 a 220= 8.

220 a 225= 5.

225 a 266= 41.

266 a 277= 11.

277 a 280= 3.

22 a 50= 28.

50 a 55= 5.

55 a 75= 20.

75 a 81= 6.

81 a 94= 13.

94 a 99= 5.

99 a 115= 16.

115 a 118= 3.

118 a 126= 8.

126 a 140= 14.

Movimientos= $4 + 3 + 25 + 37 + 8 + 5 + 41 + 11 + 3 + 28 + 5 + 20 + 6 + 13 + 5 + 16 + 3 + 8 + 14$

Movimientos= 255.

Ejercicio 33.

Se supone un disco con 300 pistas (numerados de 0 a 299), donde la cabeza se encuentra en la pista 140 atendiendo un requerimiento y, anteriormente, atendió un requerimiento en la pista 135.

Si la cola de requerimientos es: {99, 110, 42, 25, 186, 270, 50, 99, 147PF, 81, 257, 94, 133, 212, 175, 130} y, después de 30 movimientos, se incorporan los requerimientos de las pistas {85, 15PF, 202 y 288} y, después de otros 40 movimientos más, se incorporan los requerimientos de las pistas {75, 149PF, 285, 201 y 59}. Realizar los diagramas para calcular el total de movimientos de head para satisfacer estos requerimientos de acuerdo a los siguientes algoritmos de scheduling de discos:

(a) FCFS (First Come, First Served).

140 a 147PF= 7.

147PF a 99= 48.

(se incorporan los nuevos requerimientos)

99 a 15PF= 84.

(se incorporan los nuevos requerimientos)

15PF a 149PF= 134.

149PF a 110= 39.

110 a 42= 68.

42 a 25= 17.

25 a 186= 161.

186 a 270= 84.

270 a 50= 220.

50 a 99= 49.

99 a 81= 18.

81 a 257= 176.

257 a 94= 163.

94 a 133= 39.

133 a 212= 79.

212 a 175= 37.

175 a 130= 45.

130 a 85= 45.

85 a 202= 117.

202 a 288= 86.

288 a 75= 213.

75 a 285= 210.

285 a 201= 84.

201 a 59= 142.

Movimientos= 7 + 48 + 84 + 134 + 39 + 68 + 17 + 161 + 84 + 220 + 49 + 18 + 176 + 163 + 38 + 79 + 37 + 45 + 45 + 117 + 86 + 213 + 210 + 84 + 142

Movimientos= 2365.

(b) SSTF (Shortest Seek Time First).

140 a 147PF= 7.

147PF a 133= 14.

133 a 130= 3.

130 a 110= 20.

(se incorporan los nuevos requerimientos)

110 a 15PF= 95.

(se incorporan los nuevos requerimientos)

15PF a 149PF= 134.

149PF a 175= 26.

175 a 186= 11.

186 a 201= 15.

201 a 202= 1.

202 a 212= 10.

212 a 257= 45.

257 a 270= 13.

270 a 285= 15.

285 a 288= 3.

288 a 99= 189.

99 a 99= 0.

99 a 94= 5.

94 a 85= 9.

85 a 81= 4.

81 a 75= 6.

75 a 59= 16.

59 a 50= 9.

50 a 42= 8.

42 a 25= 17.

Movimientos= $7 + 14 + 3 + 20 + 95 + 134 + 26 + 11 + 15 + 1 + 10 + 45 + 13 + 15 + 3 + 189 + 0 + 5 + 9 + 4 + 6 + 16 + 9 + 8 + 17$

Movimientos= 675.

(c) *Scan*.

140 a 147PF= 7.

147PF a 175= 28.

(se incorporan los nuevos requerimientos)

175 a 15PF= 160.

(se incorporan los nuevos requerimientos)

15PF a 149PF= 134.

149PF a 186= 37.

186 a 201= 15.

201 a 202= 1.

202 a 212= 10.

212 a 257= 45.

257 a 270= 13.

270 a 285= 15.

285 a 288= 3.

288 a 299= 11.

299 a 133= 166.

133 a 130= 3.

130 a 110= 20.

110 a 99= 11.

99 a 99= 0.
 99 a 94= 5.
 94 a 85= 9.
 85 a 81= 4.
 81 a 75= 6.
 75 a 59= 16.
 59 a 50= 9.
 50 a 42= 8.
 42 a 25= 17.

Movimientos= $7 + 28 + 160 + 134 + 37 + 15 + 1 + 10 + 45 + 13 + 15 + 3 + 11 + 166 + 3$
 $+ 20 + 11 + 0 + 5 + 9 + 4 + 6 + 16 + 9 + 8 + 17$
 Movimientos= 753.

(d) *Look.*

140 a 147PF= 7.
 147PF a 175= 28.
 175 a 15PF= 160.
 15PF a 149PF= 134.
 149PF a 186= 37.
 186 a 201= 15.
 201 a 202= 1.
 202 a 212= 10.
 212 a 257= 45.
 257 a 270= 13.
 270 a 285= 15.
 285 a 288= 3.
 288 a 133= 155.
 133 a 130= 3.
 130 a 110= 20.
 110 a 99= 11.
 99 a 99= 0.
 99 a 94= 5.
 94 a 85= 9.
 85 a 81= 4.
 81 a 75= 6.
 75 a 59= 16.
 59 a 50= 9.
 50 a 42= 8.
 42 a 25= 17.

(se incorporan los nuevos requerimientos)

(se incorporan los nuevos requerimientos)

Movimientos= $7 + 28 + 160 + 134 + 37 + 15 + 1 + 10 + 45 + 13 + 15 + 3 + 155 + 3 + 20$
 $+ 11 + 0 + 5 + 9 + 4 + 6 + 16 + 9 + 8 + 17$
 Movimientos= 735.

(e) *C-Scan (Circular Scan).*

140 a 147PF= 7.
 147PF a 175= 28. (se incorporan los nuevos requerimientos)
 175 a 15PF= 160. (se incorporan los nuevos requerimientos)
 15PF a 149PF= 134.
 149PF a 186= 37.
 186 a 201= 15.
 201 a 202= 1.
 202 a 212= 10.
 212 a 257= 45.
 257 a 270= 13.
 270 a 285= 15.
 285 a 288= 3.
 288 a 299= 11.
 0 a 25= 25.
 25 a 42= 17.
 42 a 50= 8.
 50 a 59= 9.
 59 a 75= 16.
 75 a 81= 6.
 81 a 85= 4.
 85 a 94= 9.
 94 a 99= 5.
 99 a 99= 0.
 99 a 110= 11.
 110 a 130= 20.
 130 a 133= 3.

Movimientos= $7 + 28 + 160 + 134 + 37 + 15 + 1 + 10 + 45 + 13 + 15 + 3 + 11 + 25 + 17$
 $+ 8 + 9 + 16 + 6 + 4 + 9 + 5 + 0 + 11 + 20 + 3$
 Movimientos= 612.

(f) C-Look (Circular Look).

140 a 147PF= 7.
 147PF a 175= 28. (se incorporan los nuevos requerimientos)
 175 a 15PF= 160. (se incorporan los nuevos requerimientos)
 15PF a 149PF= 134.
 149PF a 186= 37.
 186 a 201= 15.
 201 a 202= 1.
 202 a 212= 10.
 212 a 257= 45.
 257 a 270= 13.
 270 a 285= 15.
 285 a 288= 3.
 25 a 42= 17.

42 a 50= 8.

50 a 59= 9.

59 a 75= 16.

75 a 81= 6.

81 a 85= 4.

85 a 94= 9.

94 a 99= 5.

99 a 99= 0.

99 a 110= 11.

110 a 130= 20.

130 a 133= 3.

Movimientos= $7 + 28 + 160 + 134 + 37 + 15 + 1 + 10 + 45 + 13 + 15 + 3 + 17 + 8 + 9 + 16 + 6 + 4 + 9 + 5 + 0 + 11 + 20 + 3$

Movimientos= 576.

PARTE IV: Administración de Archivos.**Ejercicio 34.**

Dadas las siguientes técnicas de administración de espacio de un archivo:

- *Asignación contigua.*
- *Asignación enlazada.*
- *Asignación indexada.*

(a) *Describir cómo trabaja cada una.*

(b) *Citar ventajas y desventajas de cada una.*

(c) *Indicar, para cada técnica, si la misma puede producir fragmentación interna y/o externa.*

A continuación, se describen las técnicas de administración de espacio de un archivo:

1. Asignación contigua:

- Cada archivo ocupa bloques consecutivos en el disco.
- Se almacena el bloque inicial y el tamaño.
- La lectura es secuencial y, extremadamente, rápida.
- Ventajas: excelente performance en acceso secuencial y aleatorio; fácil de administrar.
- Desventajas: necesita conocer el tamaño del archivo por adelantado; difícil crecimiento del archivo.
- Puede producir fragmentación interna (último bloque puede quedar parcialmente lleno) y fragmentación externa (requiere bloques contiguos).

2. Asignación enlazada:

- El archivo se guarda como una lista enlazada de bloques en cualquier parte del disco.
- Cada bloque contiene un puntero al siguiente bloque.
- No requiere espacio contiguo.
- Ventajas: archivos pueden crecer dinámicamente; no requiere bloques contiguos.
- Desventajas: acceso aleatorio muy lento (hay que recorrer la lista); *overhead* por punteros; si se corrompe un puntero, se pierde parte del archivo.
- Puede producir fragmentación interna (cada bloque puede no usarse por completo).

3. Asignación indexada:

- Se crea un bloque índice (*index block*) para cada archivo.
- El bloque índice contiene una tabla de punteros a los bloques de datos.
- Los bloques pueden estar dispersos como en la asignación enlazada, pero se accede directo mediante el índice.
- Ventajas: permite acceso directo y secuencial; archivos pueden crecer sin necesidad de contigüidad; no depende de la integridad de una cadena de punteros, como en la asignación enlazada.

- Desventajas: ocupa espacio extra para el bloque índice; el tamaño del archivo puede estar limitado por la cantidad de punteros del índice; ligeramente, más compleja de administrar.
- Puede producir fragmentación interna (tamaño del último bloque + tamaño del bloque índice).

Ejercicio 35: Gestión de Espacio Libre.

Dados los siguientes métodos de gestión de espacio libre en un disco:

- *Tabla de bits.*
- *Lista enlazada.*
- *Agrupamiento.*
- *Recuento.*

(a) *Describir cómo trabajan estos métodos.*

(b) *Citar ventajas y desventajas de cada uno.*

A continuación, se describen cómo trabajan estos métodos:

1. Tabla de bits:

- Se mantiene un vector de bits, donde cada bit representa un bloque del disco: 0 es bloque libre y 1 es bloque ocupado.
- El *bitmap* suele mantenerse en memoria para consultas rápidas.
- Ventajas: muy fácil identificar bloques contiguos (útil para asignación contigua); ocupa poco espacio (1 bit por bloque); rápida búsqueda usando operaciones de bits.
- Desventajas: si el disco es grande, el *bitmap* puede ser, relativamente, extenso; puede requerir leer varias palabras de memoria para encontrar un rango libre.

2. Lista enlazada:

- Todos los bloques libres están encadenados uno tras otro formando una lista.
- Cada bloque libre contiene un puntero al siguiente bloque libre.
- El *superblock* mantiene la dirección del primer bloque libre.
- Ventajas: sencilla de implementar; no requiere espacio en memoria más que un puntero al primer bloque.
- Desventajas: ineficiente para buscar bloques contiguos; acceso lento (cada búsqueda requiere leer bloques libres del disco); si se corrompe un puntero, se pierde parte de la lista.

3. Agrupamiento:

- Variante de la lista enlazada.
- En lugar de que cada bloque libre apunte a uno solo, un bloque libre contiene la dirección de los siguientes N bloques libres y un puntero a otro bloque que contiene el siguiente grupo.
- Se reducen accesos al disco.
- Ventajas: búsqueda mucho más rápida que en la lista simple; menos accesos al disco para localizar bloques libres; sigue siendo simple de implementar.
- Desventajas: todavía es difícil encontrar grandes áreas contiguas; depende de la integridad de los bloques de grupo.

4. Recuento:

- Se observa que los bloques libres suelen aparecer en grupos contiguos.
- Se guarda una lista de entradas de la forma: (bloque inicial, cantidad de bloques contiguos libres).
- Permite representar grandes rangos de bloques libres con un solo registro.

- Ventajas: excelente para sistemas en los que los bloques libres están agrupados; permite encontrar áreas contiguas rápidamente; representa muchos bloques libres con pocos registros; muy eficiente en sistemas grandes.
- Desventajas: más complejo de mantener cuando los archivos se crean y borran continuamente; menos eficiente si los bloques libres están muy fragmentados y no contiguos.

Ejercicio 36.

Dado un disco con las características vistas en el Ejercicio 28:

- 7 platos con 2 caras utilizables cada uno.
- 1.100 cilindros, con 300 sectores por pista, donde cada sector es de 512 bytes.
- Seek time de 10 ms.
- 9.000 RPM.
- Velocidad de transferencia de 10 MiB/s (MebiBytes por segundo).

(a) Calcular el tiempo de transferencia real de un archivo de 15 MiB (MebiBytes) almacenado en el disco de manera secuencial (todos sus bloques almacenados de manera consecutiva).

$$\text{Sectores} = \frac{15 \times 1048576 \text{ bytes}}{512 \text{ bytes}}$$

$$\text{Sectores} = \frac{15728640 \text{ bytes}}{512 \text{ bytes}}$$

$$\text{Sectores} = 30720.$$

$$\text{Latencia} = 0,5 \frac{1}{\frac{9000}{60 \times 1000}} \text{ ms}$$

$$\text{Latencia} = 0,5 \frac{1}{\frac{9000}{60000}} \text{ ms}$$

$$\text{Latencia} = 0,5 \frac{1}{0,15} \text{ ms}$$

$$\text{Latencia} = 0,5 * 6, \hat{6} \text{ ms}$$

$$\text{Latencia} = 3, \hat{3} \text{ ms}.$$

$$\text{Tiempo Transferencia} = \frac{512 \text{ bytes}}{\frac{10 \times 1048576 \text{ bytes}}{1000}} \text{ ms}$$

$$\text{Tiempo Transferencia} = \frac{512 \text{ bytes}}{\frac{10485760 \text{ bytes}}{1000}} \text{ ms}$$

$$\text{Tiempo Transferencia} = \frac{512 \text{ bytes}}{1048,56 \text{ bytes}} \text{ ms}$$

$$\text{Tiempo Transferencia} = 0,48828125 \text{ ms}.$$

$$\text{Tiempo Transferencia Real} = 10 \text{ ms} + 3, \hat{3} \text{ ms} + 0,48828125 \text{ ms} * 30720$$

$$\text{Tiempo Transferencia Real} = 10 \text{ ms} + 3, \hat{3} \text{ ms} + 1500 \text{ ms}$$

$$\text{Tiempo Transferencia Real} = 1513, \hat{3} \text{ ms}.$$

Por lo tanto, el tiempo de transferencia real de un archivo de 15 MiB almacenado en el disco de manera secuencial es 1.513,3 ms.

(b) Calcular el tiempo de transferencia real de un archivo de 16 MiB (MebiBytes) almacenado en el disco de manera aleatoria.

$$\text{Sectores} = \frac{16 \times 1048576 \text{ bytes}}{512 \text{ bytes}}$$

$$\text{Sectores} = \frac{16777216 \text{ bytes}}{512 \text{ bytes}}$$
$$\text{Sectores} = 30768.$$

$$\text{Tiempo Transferencia Real} = (10 \text{ ms} + 3,3 \text{ ms} + 0,048828125 \text{ ms}) * 30768$$

$$\text{Tiempo Transferencia Real} = 13,3821614583 \text{ ms} * 30768$$

$$\text{Tiempo Transferencia Real} = 411742,34375 \text{ ms}.$$

Por lo tanto, el tiempo de transferencia real de un archivo de 16 MiB almacenado en el disco de manera aleatoria es 411.724,34375 ms.

Ejercicio 37: Unidad de Asignación.

Las técnicas vistas pueden referenciar a sectores físicos de medio de almacenamiento o clusters (conjunto consecutivo de sectores físicos). Citar ventajas y desventajas de la utilización de cada forma de unidad de asignación.

A continuación, se citan ventajas y desventajas de la utilización de cada forma de unidad de asignación:

1. Sectores físicos de medio de almacenamiento:

- Ventajas: menor fragmentación interna, ya que se desperdicia menos espacio por archivo; asignación más precisa (los archivos usan sólo los sectores exactos que necesitan); mejor para archivos pequeños (no desperdician espacio); control más fino sobre la distribución en disco.
- Desventajas: mayor sobrecarga en las estructuras del sistema de archivos (más entradas por archivo); más operaciones de E/S por archivo (muchos sectores a leer/escribir); mayor tiempo para buscar sectores libres, especialmente si están dispersos; el sistema se vuelve más complejo y más lento en archivos grandes.

2. Clusters:

- Ventajas: menor sobrecarga administrativa; lectura/escritura más rápida (se accede a grandes porciones contiguas); ideal para archivos grandes, ya que mejora el rendimiento; simplifica la búsqueda de espacio libre.
- Desventajas: puede generar fragmentación interna; desperdicio notable en sistemas con muchos archivos pequeños; menor granularidad (asignación menos precisa).

Ejercicio 38.

Si se tiene un disco con 10 platos con 2 caras utilizables cada uno (20 cabezas), 350 cilindros y con 500 sectores por pista, ¿cuántos clusters puede haber si se toma un tamaño de 4 sectores? ¿Y si el cluster es de 8 sectores?

Sectores= sectores/pista * cilindros * cabezas

Sectores= 500 * 350 * 20

Sectores= 3500000.

Con tamaño de 4 sectores:

Clusters= $\frac{3500000}{4}$

Clusters= 875000.

Con tamaño de 8 sectores:

Clusters= $\frac{3500000}{8}$

Clusters= 437500.

Por lo tanto, si se tiene un disco con 10 platos con 2 caras utilizables cada uno (20 cabezas), 350 cilindros y con 500 sectores por pista, si se toma un tamaño de 4 sectores, puede haber 875.000 *clusters* y, si el *cluster* es de 8 sectores, puede haber 437.500 *clusters*.

Ejercicio 39.

Asumiendo un sistema de archivos que utiliza, como unidad de asignación, un cluster de 8 sectores de disco y que el primer cluster de datos (cluster 0) se comienza a almacenar en el sector LBA 120. Determinar el o los valores LBA de los sectores que corresponden a los siguientes números de cluster:

(a) 0.

$$\text{LBA inicial} = 120 + (0 * 8)$$

$$\text{LBA inicial} = 120 + 0$$

$$\text{LBA inicial} = 120.$$

Por lo tanto, los valores LBA de los sectores que corresponden al *cluster* 0 son de 120 a 127.

(b) 123.

$$\text{LBA inicial} = 120 + (123 * 8)$$

$$\text{LBA inicial} = 120 + 984$$

$$\text{LBA inicial} = 1104.$$

Por lo tanto, los valores LBA de los sectores que corresponden al *cluster* 123 son de 1.104 a 1.111.

(c) 299.

$$\text{LBA inicial} = 120 + (299 * 8)$$

$$\text{LBA inicial} = 120 + 2392$$

$$\text{LBA inicial} = 2512.$$

Por lo tanto, los valores LBA de los sectores que corresponden al *cluster* 299 son de 2.512 a 2.519.

(d) 1001.

$$\text{LBA inicial} = 120 + (1001 * 8)$$

$$\text{LBA inicial} = 120 + 8008$$

$$\text{LBA inicial} = 8128.$$

Por lo tanto, los valores LBA de los sectores que corresponden al *cluster* 1.001 son de 8.128 a 8.135.

Ejercicio 40.

Describir la organización del FileSystem de Unix System V. Tener en cuenta la organización del mismo, estructuras de datos, etc.

A continuación, se describe la organización del *File System* de Unix *System V*:

1. Boot Block: Código de arranque.
2. Superblock: Metadatos globales del *File System*.
3. i-list: Tabla fija de i-nodos.
4. Directorios: Mapean nombres a i-nodos.
5. Data Blocks: Contienen datos reales.
6. Free List: Administración de espacio libre.
7. Estructuras en memoria: Tabla de i-nodos, tabla de archivos abiertos, *buffer* caché.

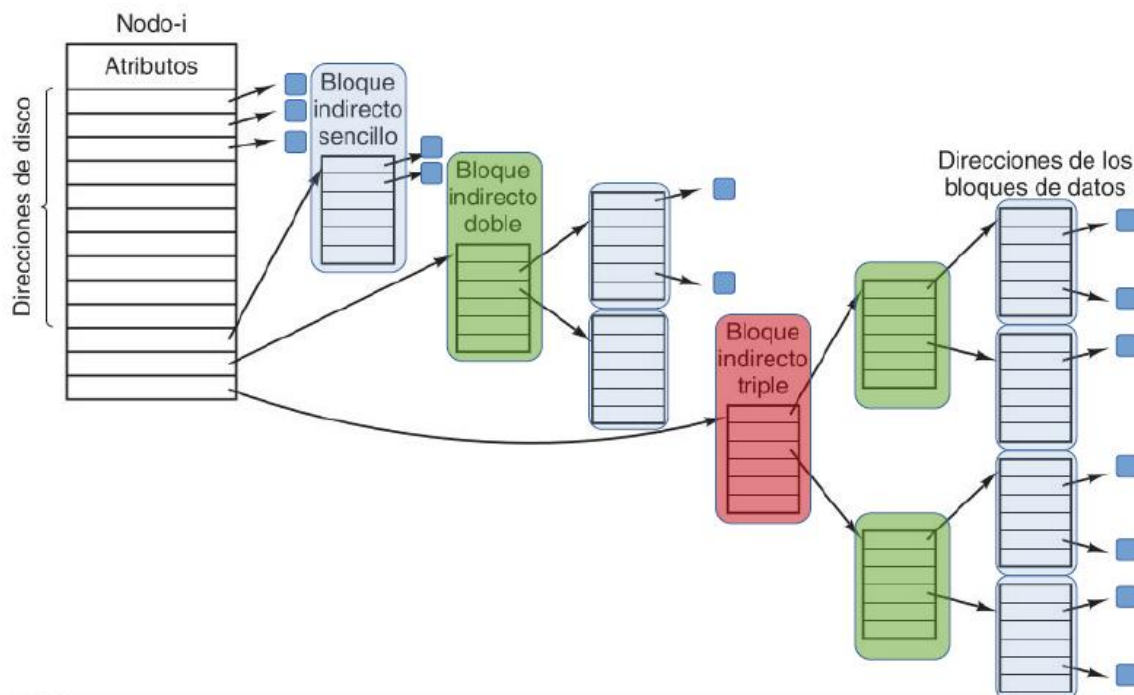
Ejercicio 41: Gestión de Archivos en UNIX System V.

El sistema de archivos de UNIX utiliza una variante del esquema de asignación indexada para la administración de espacio de los archivos. Cada archivo o directorio está representado por una estructura que mantiene, entre otra información, las direcciones de los bloques que contienen los datos del archivo: el I-NODO.

Cada I-NODO contiene 13 campos para direcciones a los bloques de datos, organizadas de la siguiente manera:

- 10 de direccionamiento directo.
- 1 de direccionamiento indirecto simple.
- 1 de direccionamiento indirecto doble.
- 1 de direccionamiento indirecto triple.

(a) Realizar un gráfico que represente la estructura del I-NODO y de los bloques de datos. Cada cluster es de 1 Kib (Kibibits).



(b) Si cada dirección para referenciar un bloque es de 32 bits:

(i) ¿Cuántas referencias (direcciones) a bloque puede contener un cluster?

$$\text{Direcciones} = \frac{1024 \text{ bits}}{32 \text{ bits}}$$

$$\text{Direcciones} = 32.$$

Por lo tanto, un *cluster* puede contener 32 referencias (direcciones) a bloque.

(ii) *¿Cuál sería el tamaño máximo de un archivo?*

$$\text{Tamaño máximo} = (10 + 32 + 32^2 + 32^3) * \frac{1024}{8} \text{ bytes}$$

$$\text{Tamaño máximo} = (10 + 32 + 1024 + 32768) * 128 \text{ bytes}$$

$$\text{Tamaño máximo} = 33834 * 128 \text{ bytes}$$

$$\text{Tamaño máximo} = 4330752 \text{ bytes.}$$

Por lo tanto, el tamaño máximo de un archivo sería 4.330.752 bytes.

Ejercicio 42.

Dado el FileSystem de UNIX System V (visto en clase), indicar qué cambios se producen en el mismo dadas las siguientes situaciones:

(a) *Se agrega un nuevo archivo “Archivo1” en el directorio “Directorio1”.*

Los cambios que se producen en el *FileSystem* de UNIX System V son:

- Se asigna un nuevo inodo para “Archivo1”.
- Se agrega una entrada (nombre + número de inodo) al directorio.
- Se actualiza el inodo del directorio (*mtime/ctime/size*).
- Se actualiza mapa de inodos.
- Si el archivo agregado no entra en los bloques existentes:
 - Se asigna uno o más bloques nuevos para el directorio.
 - Se actualiza mapa de bloques.

(b) *Se agrega nuevo contenido al archivo “Archivo1”.*

Los cambios que se producen en el *FileSystem* de UNIX System V son:

- Se modifica el inodo de “Archivo1”.
- Se actualiza el inodo del directorio (*mtime*).
- Si el contenido agregado al archivo no entra en los bloques existentes:
 - Se asigna uno o más bloques nuevos para el directorio.
 - Se actualiza mapa de bloques.

(c) *Se modifica el nombre del archivo “Archivo1” a “Archivo2”.*

Los cambios que se producen en el *FileSystem* de UNIX System V son:

- Se modifica la entrada del directorio (nombre de inodo).
- Se actualiza el inodo del directorio (*mtime/ctime*).

(d) *Se hace un link duro a “Archivo2” en el mismo directorio con nombre “LinkArchivo2”.*

Los cambios que se producen en el *FileSystem* de UNIX System V son:

- Se agrega una entrada (nombre + número de inodo) al directorio.
- Se actualiza el inodo del directorio (*mtime/ctime/size*).
- Se actualiza el inodo del archivo original (se incrementa el *link count*).

- Si no hay espacio disponible para una entrada extra:
 - Se asigna un nuevo bloque para el directorio.
 - Se actualiza mapa de bloques.

(e) *Se borra un archivo “Archivo3”.*

Los cambios que se producen en el *FileSystem* de UNIX *System V* son:

- Se elimina la entrada (nombre + número de inodo) del directorio.
- Se actualiza el inodo del directorio (*mtime/ctime/size*).
- Se actualiza el inodo del archivo original (se decrementa el *link count*).
- Si *link count* llega a cero:
 - Se libera uno o más bloques del archivo.
 - Se libera el inodo del archivo.
 - Se actualizan mapa de bloques y mapa de inodos.

Ejercicio 43.

Se tiene un FileSystem con un modelo indexado con niveles de indirección, donde se mantiene un i-nodo con 10 índices de direccionamiento directo, 1 índice para indirecto simple y 2 para indirecto doble. Considerando, además, que se utilizan 32 bits para referenciar direcciones a bloques/clusters.

Suponer que existen los siguientes archivos con sus tamaños lógicos:

- ArchivoA: de 55 KibiBytes.
- ArchivoB: de 12 KibiBytes.
- ArchivoC: de 1.075 KibiBytes.

(a) Si se utiliza, como unidad de asignación, el sector de disco de 512 bytes:

(i) ¿Cuántas direcciones a sectores puede contener un sector de disco?

$$\begin{aligned}\text{Direcciones} &= \frac{512 \text{ bytes}}{\frac{2^{32} \text{ bytes}}{8}} \\ \text{Direcciones} &= \frac{512 \text{ bytes}}{4 \text{ bytes}} \\ \text{Direcciones} &= 128.\end{aligned}$$

Por lo tanto, un sector de disco puede contener 128 direcciones a sectores.

(ii) ¿Cuál sería el tamaño máximo que un archivo podría alcanzar?

$$\begin{aligned}\text{Tamaño máximo} &= (10 + 128 + 2 * 128^2) * 512 \text{ bytes} \\ \text{Tamaño máximo} &= (10 + 128 + 2 * 16384) * 512 \text{ bytes} \\ \text{Tamaño máximo} &= (10 + 128 + 32768) * 512 \text{ bytes} \\ \text{Tamaño máximo} &= 32906 * 512 \text{ bytes} \\ \text{Tamaño máximo} &= 16847872 \text{ bytes}.\end{aligned}$$

Por lo tanto, el tamaño máximo que un archivo podrá alcanzar sería 16.847.872 bytes (16.453 KiB).

(iii) ¿Cuántos sectores se necesitan para almacenar cada uno de los archivos antes enumerados? Indicar el tamaño físico que se requirió para almacenar cada uno.

$$\begin{aligned}\text{Sectores}_A &= \frac{55 * 1024 \text{ bytes}}{512 \text{ bytes}} \\ \text{Sectores}_A &= 55 * 2 \\ \text{Sectores}_A &= 110.\end{aligned}$$

$$\text{Sectores}_B = \frac{12 * 1024 \text{ bytes}}{512 \text{ bytes}}$$

$$Sectores_B = 12 * 2$$

$$Sectores_B = 24.$$

$$Sectores_C = \frac{1075 * 1024 \text{ bytes}}{512 \text{ bytes}}$$

$$Sectores_C = 1075 * 2$$

$$Sectores_C = 2150.$$

$$TF_A = 110 * 512 \text{ bytes}$$

$$TF_A = 56320 \text{ bytes}$$

$$TF_A = 55 \text{ KiB.}$$

$$TF_B = 24 * 512 \text{ bytes}$$

$$TF_B = 12288 \text{ bytes}$$

$$TF_B = 12 \text{ KiB.}$$

$$TF_C = 2150 * 512 \text{ bytes}$$

$$TF_C = 1100800 \text{ bytes}$$

$$TF_C = 1075 \text{ KiB.}$$

Por lo tanto, para almacenar los archivos ArchivoA, ArchivoB y ArchivoC, se necesitan 110, 24 y 2.150 sectores, respectivamente.

(iv) Indicar la fragmentación interna que se produce para cada uno de los archivos antes enumerados.

$$FI_A = TF_A - TL_A$$

$$FI_A = 55 \text{ KiB} - 55 \text{ KiB}$$

$$FI_A = 0 \text{ KiB.}$$

$$FI_B = TF_B - TL_B$$

$$FI_B = 12 \text{ KiB} - 12 \text{ KiB}$$

$$FI_B = 0 \text{ KiB.}$$

$$FI_C = TF_C - TL_C$$

$$FI_C = 1075 \text{ KiB} - 1075 \text{ KiB}$$

$$FI_C = 0 \text{ KiB.}$$

(b) Si se utiliza, como unidad de asignación, el cluster de disco de 2 KibiBytes:

(i) ¿Cuántas direcciones a clusters puede contener un cluster de disco?

$$\text{Direcciones} = \frac{2 * 1024 \text{ bytes}}{\frac{32}{8} \text{ bytes}}$$

$$\text{Direcciones} = \frac{2048 \text{ bytes}}{4 \text{ bytes}}$$

$$\text{Direcciones} = 512.$$

Por lo tanto, un *cluster* de disco puede contener 512 direcciones a *clusters*.

(ii) ¿Cuál sería el tamaño máximo que un archivo podría alcanzar?

$$\text{Tamaño máximo} = (10 + 512 + 2 * 512^2) * 2 * 1024 \text{ bytes}$$

$$\text{Tamaño máximo} = (10 + 512 + 2 * 262144) * 2048 \text{ bytes}$$

$$\text{Tamaño máximo} = (10 + 512 + 524288) * 2048 \text{ bytes}$$

$$\text{Tamaño máximo} = 524810 * 2048 \text{ bytes}$$

$$\text{Tamaño máximo} = 1074810880 \text{ bytes.}$$

Por lo tanto, el tamaño máximo que un archivo podrá alcanzar sería 1.074.810.880 bytes (1.049.620 KiB).

(iii) ¿Cuántos *clusters* se necesitan para almacenar cada uno de los archivos antes enumerados? Indicar el tamaño físico que se requirió para almacenar cada uno.

$$\text{Clusters}_A = \frac{55 * 1024 \text{ bytes}}{2048 \text{ bytes}}$$

$$\text{Clusters}_A = 55 * 0,5$$

$$\text{Clusters}_A = 27,5$$

$$\text{Clusters}_A = 28.$$

$$\text{Clusters}_B = \frac{12 * 1024 \text{ bytes}}{2048 \text{ bytes}}$$

$$\text{Clusters}_B = 12 * 0,5$$

$$\text{Clusters}_B = 6.$$

$$\text{Clusters}_C = \frac{1075 * 1024 \text{ bytes}}{2048 \text{ bytes}}$$

$$\text{Clusters}_C = 1075 * 0,5$$

$$\text{Clusters}_C = 537,5$$

$$\text{Clusters}_C = 538.$$

$$TF_A = 28 * 2048 \text{ bytes}$$

$$TF_A = 57344 \text{ bytes}$$

$$TF_A = 56 \text{ KiB.}$$

$$TF_B = 6 * 2048 \text{ bytes}$$

$$TF_B = 12288 \text{ bytes}$$

$$TF_B = 12 \text{ KiB.}$$

$$TF_C = 538 * 2048 \text{ bytes}$$

$$TF_C = 1101824 \text{ bytes}$$

$$TF_C = 1076 \text{ KiB.}$$

Por lo tanto, para almacenar los archivos ArchivoA, ArchivoB y ArchivoC, se necesitan 28, 6 y 538 *clusters*, respectivamente.

(iv) Indicar la fragmentación interna que se produce para cada uno de los archivos antes enumerados.

$$FI_A = TF_A - TL_A$$

$$FI_A = 56 \text{ KiB} - 55 \text{ KiB}$$

$$FI_A = 1 \text{ KiB.}$$

$$FI_B = TF_B - TL_B$$

$$FI_B = 12 \text{ KiB} - 12 \text{ KiB}$$

$$FI_B = 0 \text{ KiB.}$$

$$FI_C = TF_C - TL_C$$

$$FI_C = 1076 \text{ KiB} - 1075 \text{ KiB}$$

$$FI_C = 1 \text{ KiB.}$$

Ejercicio 44.

Para los incisos (a) y (b) el ejercicio anterior y considerando que el FileSystem se utiliza en un disco con las siguientes características:

- 6 platos con 2 caras útiles cada uno.
- 2.500 pistas por cara y 100 sectores por pista de 512 bytes cada uno.
- 7.200 RPM.
- Seek time de 10,5 ms.
- Velocidad de transferencia de 146 MiB/s (MebiBytes por segundo).

Calcular el tiempo de transferencia de los 3 archivos (ArchivoA, ArchivoB y ArchivoC) del ejercicio previo.

(a)

$$\begin{aligned}\text{Velocidad de transferencia} &= \frac{146 \cdot 1024}{1000} \text{ KiB/ms} \\ \text{Velocidad de transferencia} &= \frac{149504 \text{ KiB}}{1000} \text{ KiB/ms} \\ \text{Velocidad de transferencia} &= 149,504 \text{ KiB/ms.}\end{aligned}$$

$$\begin{aligned}\text{Tiempo Transferencia}_A &= \frac{55 \text{ KiB}}{149,504 \text{ KiB}} \text{ ms} \\ \text{Tiempo Transferencia}_A &= 0,368 \text{ ms.}\end{aligned}$$

$$\begin{aligned}\text{Tiempo Transferencia}_B &= \frac{12 \text{ KiB}}{149,504 \text{ KiB}} \text{ ms} \\ \text{Tiempo Transferencia}_B &= 0,08 \text{ ms.}\end{aligned}$$

$$\begin{aligned}\text{Tiempo Transferencia}_C &= \frac{1075 \text{ KiB}}{149,504 \text{ KiB}} \text{ ms} \\ \text{Tiempo Transferencia}_C &= 7,19 \text{ ms.}\end{aligned}$$

Por lo tanto, el tiempo de transferencia de los archivos ArchivoA, ArchivoB y ArchivoC es 0,368 ms, 0,08 ms y 7,19 ms, respectivamente.

(b)

$$\begin{aligned}\text{Velocidad de transferencia} &= \frac{146 \cdot 1024}{1000} \text{ KiB/ms} \\ \text{Velocidad de transferencia} &= \frac{149504 \text{ KiB}}{1000} \text{ KiB/ms} \\ \text{Velocidad de transferencia} &= 149,504 \text{ KiB/ms.}\end{aligned}$$

$$\begin{aligned}\text{Tiempo Transferencia}_A &= \frac{56 \text{ KiB}}{149,504 \text{ KiB}} \\ \text{Tiempo Transferencia}_A &= 0,375 \text{ ms.}\end{aligned}$$

$$\text{Tiempo Transferencia}_A = \frac{12 \text{ KiB}}{149,504 \text{ KiB}}$$

Tiempo Transferencia_B = 0,08 ms.

$$\textit{Tiempo Transferencia}_C = \frac{1076 \text{ KiB}}{149,504 \text{ KiB}} \text{ ms}$$

Tiempo Transferencia_C = 7,197 ms.

Por lo tanto, el tiempo de transferencia de los archivos ArchivoA, ArchivoB y ArchivoC es 0,375 ms, 0,08 ms y 7,197 ms, respectivamente.

Ejercicio 45.

Analizar las siguientes fórmulas necesarias para localizar un I-NODO en la tabla de inodos de un FileSystem similar a System V (visto en la teoría):

- *Número de bloque del inodo = $[(\text{nro. de inodo} - 1) / \text{nro. de inodos por bloque}] + \text{bloque de comienzo de la lista de inodos}$.*
- *Desplazamiento en el bloque del inodo = $[(\text{nro. de inodo} - 1) \bmod (\text{número de inodos por bloque})] * \text{medida de inodo del disco}$.*

(a) *Según la primera fórmula, asumiendo que, en el bloque 2, está en el comienzo de la lista de inodos y que hay 8 inodos por bloque, calcular dónde se encuentra el inodo 8 y el 9. ¿Dónde estarían para un bloque de disco con 16 inodos?*

Con 8 inodos por bloque:

$$\text{Número bloque}_{\text{inodo } 8} = \frac{8-1}{8} + 2$$

$$\text{Número bloque}_{\text{inodo } 8} = \frac{7}{8} + 2$$

$$\text{Número bloque}_{\text{inodo } 8} = 0 + 2$$

$$\text{Número bloque}_{\text{inodo } 8} = 2.$$

$$\text{Número bloque}_{\text{inodo } 9} = \frac{9-1}{8} + 2$$

$$\text{Número bloque}_{\text{inodo } 9} = \frac{8}{8} + 2$$

$$\text{Número bloque}_{\text{inodo } 9} = 1 + 2$$

$$\text{Número bloque}_{\text{inodo } 9} = 3.$$

Con 16 inodos por bloque:

$$\text{Número bloque}_{\text{inodo } 8} = \frac{8-1}{16} + 2$$

$$\text{Número bloque}_{\text{inodo } 8} = \frac{7}{16} + 2$$

$$\text{Número bloque}_{\text{inodo } 8} = 0 + 2$$

$$\text{Número bloque}_{\text{inodo } 8} = 2.$$

$$\text{Número bloque}_{\text{inodo } 9} = \frac{9-1}{16} + 2$$

$$\text{Número bloque}_{\text{inodo } 9} = \frac{8}{16} + 2$$

$$\text{Número bloque}_{\text{inodo } 9} = 0 + 2$$

$$\text{Número bloque}_{\text{inodo } 9} = 2.$$

Por lo tanto, si hay 8 inodos por bloque, los inodos 8 y 9 se encuentran en los bloques 2 y 3, respectivamente, mientras que, para un bloque de disco con 16 inodos, estarían en los bloques 2 y 2, respectivamente.

(b) *De acuerdo a la segunda fórmula, si cada inodo del disco ocupa 64 bytes y hay 8 inodos por bloque de disco, el inodo 8 comienza en el desplazamiento 448 del bloque de*

disco. ¿Dónde empieza el 6? Si fueran inodos de 128 bytes y 24 inodos por bloque, ¿dónde empezaría el inodo 8?

Desplazamiento bloque_{inodo 6} = $[(6 - 1) \text{ MOD } 8] * 64 \text{ bytes}$

Desplazamiento bloque_{inodo 6} = $(5 \text{ MOD } 8) * 64 \text{ bytes}$

Desplazamiento bloque_{inodo 6} = $5 * 64 \text{ bytes}$

Desplazamiento bloque_{inodo 6} = 320 bytes.

Desplazamiento bloque_{inodo 8} = $[(8 - 1) \text{ MOD } 24] * 128 \text{ bytes}$

Desplazamiento bloque_{inodo 8} = $(7 \text{ MOD } 24) * 128 \text{ bytes}$

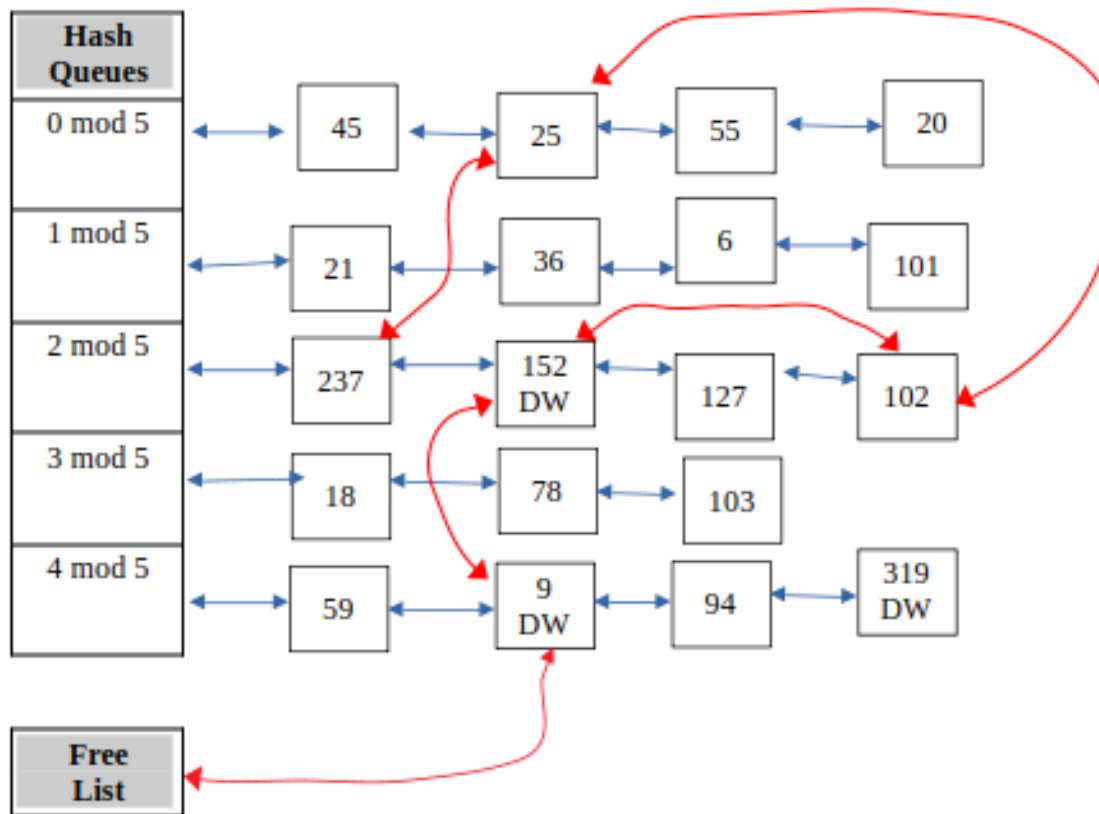
Desplazamiento bloque_{inodo 8} = $7 * 128 \text{ bytes}$

Desplazamiento bloque_{inodo 8} = 896 bytes.

Por lo tanto, si cada inodo del disco ocupa 64 bytes y hay 8 inodos por bloque de disco, el inodo 6 empieza en el desplazamiento 320 bytes del bloque de disco y, si fueran inodos de 128 bytes y 24 inodos por bloque, el inodo 8 empezaría en el desplazamiento 896 bytes del bloque de disco.

PARTE V: Caché de Disco.**Ejercicio 46.**

Dado el siguiente gráfico que representa el estado de Buffer Caché en Unix System V (visto en clase):



Responder a los siguientes incisos (considerar cambios en los headers, las hash queue, free list, operaciones de E/S, etc.):

(a) ¿Qué sucedería si un proceso P1 requiere el bloque 45?

Escenario 5: El Kernel encuentra el bloque en la hash queue, pero está busy.

Si un proceso P1 solicita el bloque 45, lo que sucedería es que el Kernel encontraría este bloque en la hash queue, pero el buffer asociado a este bloque está marcado como busy. El proceso P1 quedaría bloqueado en espera a que el buffer se desbloquee. Cuando el proceso que lo tiene lo libere, P1 y otros procesos en espera serían despertados, y el proceso P1 deberá buscar, nuevamente, en la hash queue y en la free list.

(b) ¿Qué sucedería si un proceso P1 solicita el bloque 113?

Escenario 2: El *Kernel* no encuentra el bloque en la *hash queue* y utiliza un *buffer* libre.

Si un proceso P1 solicita el bloque 113, lo que sucedería es que el *Kernel* no encontraría este bloque en la *hash queue* y se tomaría el primer *buffer* de la *free list* (el menos recientemente usado). El *Kernel* removería el *buffer* de la *free list* y cambiaría su estado a *busy*. El *Kernel* realizaría una operación de lectura desde el disco para cargar el bloque 113 en el *buffer* obtenido. El proceso P1 quedaría bloqueado en espera a que finalice dicha operación de E/S. Una vez finalizada esta operación de E/S, el *Kernel* asignaría el *buffer* al proceso P1, insertaría el *header* del *buffer* en la *hash queue* correspondiente (sólo se cambian punteros, no se intercambian ubicaciones de memoria) y, finalmente, reacomodaría los punteros de la *free list* (ya que uno de sus elementos fue retirado).

(c) ¿Qué sucedería si un proceso P1 solicita el bloque 102?

Escenario 1: El *Kernel* encuentra el bloque en la *hash queue* y el *buffer* está libre (en la *free list*).

Si un proceso P1 requiere el bloque 102, lo que sucedería es que el *Kernel* encontraría este bloque en la *hash queue* y su *buffer* estaría disponible en la *free list*. El *Kernel* removería el *buffer* asociado al bloque 102 de la *free list*, cambiaría su estado a *busy*, lo asignaría al proceso P1 y, finalmente, reacomodaría los punteros de la *free list* (ya que uno de sus elementos fue retirado).

(d) ¿Qué sucedería si un proceso P1 solicita el bloque 9?

¿Escenario 1?: El *Kernel* encuentra el bloque en la *hash queue* y el *buffer* está libre (en la *free list*), pero está marcado como DW (*Delayed Write*).

Si un proceso P1 requiere el bloque 9, lo que sucedería es que el *Kernel* encontraría este bloque en la *hash queue* y su *buffer* estaría disponible en la *free list*, pero está marcado como DW. El *Kernel* realizaría una operación de escritura del contenido del *buffer* al disco. El proceso P1 quedaría bloqueado en espera a que finalice dicha operación de E/S. Una vez finalizada esta operación de E/S, el *Kernel* eliminaría la marca DW, removería el *buffer* asociado al bloque 9 de la *free list*, cambiaría su estado a *busy*, lo asignaría al proceso P1 y, finalmente, reacomodaría los punteros de la *free list* (ya que uno de sus elementos fue retirado).